TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

1. Datos informativos

Asignatura:	Teoría de la Programación
Docente:	Jose Oswaldo Guaman Quinche
Estudiante(s):	Joel Alejandro Paccha Luzuriaga
Curso / Paralelo:	I CICLO – Computación “A”
Fecha de entrega:	Martes 04 de noviembre de 2026
Tema:	Desarrollo de algoritmos: representación gráfica, pseudocódigo, prueba de escritorio y codificación en C

2. Objetivo

Desarrollar los algoritmos propuestos por el docente aplicando las distintas formas de representación y construcción de soluciones: diagrama de flujo, pseudocódigo, prueba de escritorio y su implementación en el lenguaje de programación C.

3. Instrucciones generales

1. El trabajo se realizará de forma individual (o en grupos de [__] personas).
2. Se deben desarrollar todos los algoritmos enviados por el docente.
3. Por cada algoritmo se debe presentar la estructura indicada en el punto 4, respetando el orden establecido.
4. Los diagramas de flujo pueden elaborarse en herramientas como draw.io, Flowgorithm o Lucidchart, o a mano legible (fotografiados).
5. La codificación debe realizarse en lenguaje C, compilada y probada. Se debe incluir una captura de pantalla de la salida del programa con datos de ejemplo.
6. El documento se entregará en formato PDF a través de [medio de entrega].
7. La copia total o parcial de trabajos será sancionada según el reglamento institucional.

8.El docente podrá solicitar una sustentación oral del trabajo para verificar la autoría de las soluciones presentadas.

> 🚫 **PROHIBIDO EL USO DE INTELIGENCIA ARTIFICIAL:** queda estrictamente prohibido utilizar herramientas de IA (ChatGPT, Gemini, Copilot, DeepSeek, Claude, entre otras) para generar o resolver los diagramas de flujo, pseudocódigo, pruebas de escritorio, código en C o conclusiones. El trabajo debe ser de **elaboración 100% propia**. El incumplimiento de esta norma se calificará con **0 (cero)** y se aplicará el reglamento institucional correspondiente.

4. Estructura del trabajo (puntos a desarrollar por cada algoritmo)

- ### Enunciado del algoritmo

  108. Diseñe un algoritmo que lea una cantidad de segundos (entero) desde teclado. Use las constantes literales 60 y 3600 para representar segundos por minuto y por hora. Calcule las horas, minutos y segundos equivalentes usando expresiones aritméticas con operadores división entera y módulo. Muestre por pantalla el resultado con el formato: "X segundos equivalen a H horas, M minutos y S segundos.

- ### Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- ### Datos de entrada:

  Cantidad de segundos

- ### Procesos:

  segundos / segundos_hora
  (segundos % segundos_hora) / segundos_minuto
  segundos % segundos_minutos

- ### Datos de salida:

  Segundos, Minutos, Horas

- ### Variables:

  segundos = 0 ; →entero
  H = 0 ; →entero
  M = 0 ; →entero
  S = 0 ; →entero

- Constantes:
  segundos_minuto = 60 ; →entero
  segundos_hora = 3600 ; →entero

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, paralelogramos y flechas.

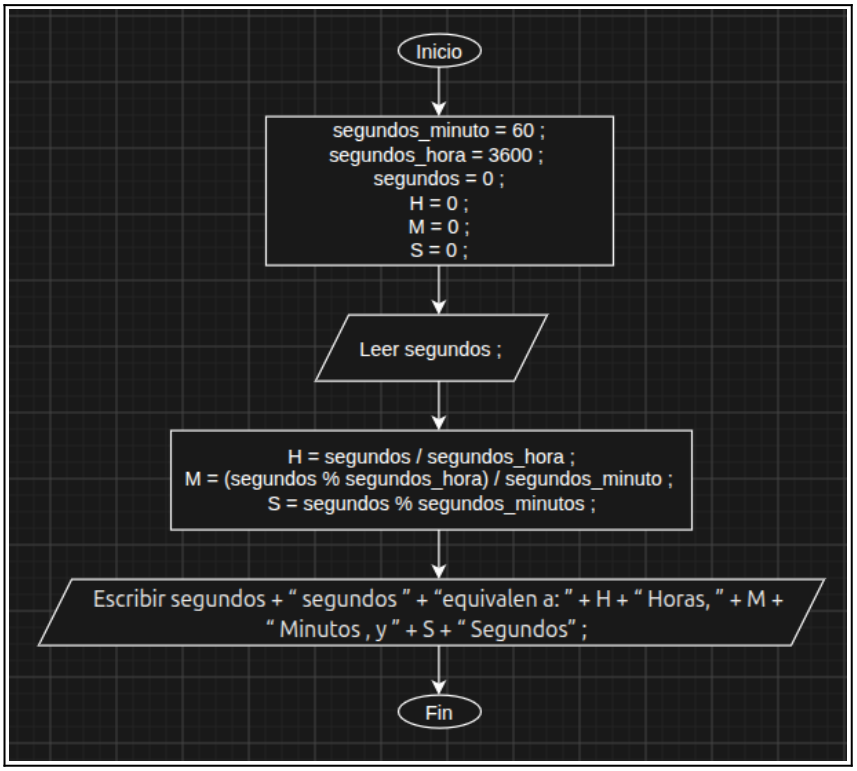


Fig.1: Diagrama de flujo en Draw.io – Algoritmo 108.

- Pseudocódigo
  Inicio
  segundos_minuto = 60 ;
  segundos_hora = 3600 ;
  segundos = 0 ;
  H = 0 ;
  M = 0 ;
  S = 0 ;
  Leer segundos ;
  H = segundos / segundos_hora ;
  M = (segundos % segundos_hora) / segundos_minuto ;
  S = segundos % segundos_minuto ;
  Escribir segundos + " segundos " + "equivalen a: " + H + " Horas, " + M + " Minutos , y " + S + " Segundos" ;

Fin

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Segundos ingresados	9876 segundos	---	---
Horas	---	9876 / 3600	2 Horas
MOD Horas	---	9876 % 3600 = 2676	---
Minutos	---	2676 / 60	44 Minutos
Segundos	---	9876 % 60	36 Segundos

Resultado: **9876 segundos equivalen a 2 Horas, 44 Minutos y 36 Segundos**

Datos	Entradas	Procesos	Salidas
Segundos ingresados	567 segundos	---	---
Horas	---	567 / 3600	0 Horas
MOD Horas	---	567 % 3600 = 567	
Minutos	---	567 / 60	9 Minutos
Segundos	---	567 % 60	27 Segundos

Resultado: **567 segundos equivalen a 0 Horas, 9 Minutos y 27 Segundos**

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 108

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

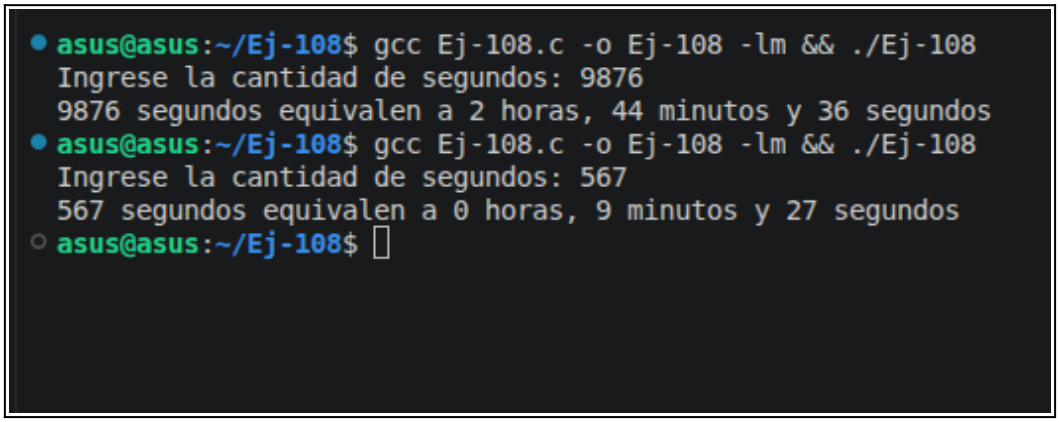


Fig.2: Resultados de compilación del programa -108.

- Enunciado del algoritmo

  134. Declare una variable entera y y una constante simbólica IVA = 1.5. Lea y desde teclado. Calcule la expresión aritmética (y % IVA) y asígnela a residuo. Evalúe la expresión lógica (residuo == 0 AND y > 0) y muestre por pantalla el mensaje correspondiente. Identifique claramente cuál es el dato de entrada y cuál es la información procesada.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Número entero

- Procesos:

  residuo = (y % IVA)

- Datos de salida:

  Resultado del residuo
  Expresión lógica (Si residuo es igual 0 y el numero ingresado mayor a 0)

- Variables:

  y = 0 ; →entero
  residuo = 0.0 ; →real

- Constantes:

  IVA = 1.5 ; →real

- Diagrama de flujo

  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

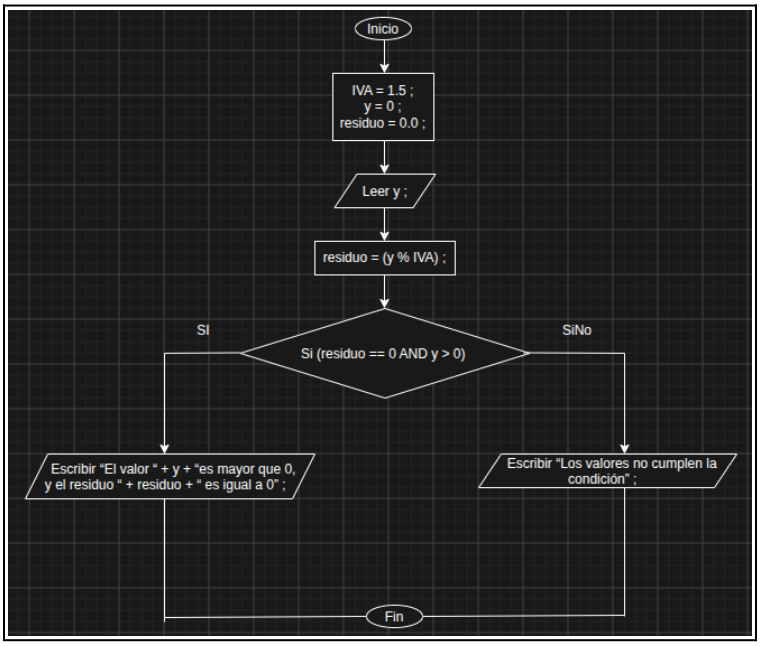


Fig.3: Diagrama de flujo en Draw.io – Algoritmo 134.

- Pseudocódigo

  Inicio
  IVA = 1.5 ;
  y = 0 ;
  residuo = 0.0 ;
  Leer y ;
  residuo = (y % IVA) ;
  Si (residuo == 0 AND y > 0)
  Escribir “El valor “ + y + “es mayor que 0, y el residuo “ + residuo + “ es igual a 0” ;
  SiNo
  Escribir “Los valores no cumplen la condición” ;
  FinSi ;
  Fin

- Prueba de escritorio

Datos	**Entradas**	**Procesos**	**Salidas**
Valor ingresado (y)	23	---	---
residuo	---	23 % 1.5	0.5
Expresión lógica	---	residuo == 0	Falso
Expresión lógica	---	y > 0	Verdadero
Resultado expresión lógica	---	residuo == 0 AND y > 0	Solo se cumple una (Falso)

Resultado: El resultado de la expresión aritmética (23 % 1.5) es 0.5, el cuál es mayor que 0 por lo que no cumple la primera condición, y aunque cumple la segunda (23 > 0), la condición general no se cumple completa por lo que es falsa.

Los valores no cumplen la condición

Datos	**Entradas**	**Procesos**	**Salidas**
Valor ingresado (y)	9	---	---
residuo	---	9 % 1.5	0
Expresión lógica	---	residuo == 0	Verdadero
Expresión lógica	---	y > 0	Verdadero
Resultado expresión lógica	---	residuo == 0 AND y > 0	Se cumplen ambas condiciones (Verdadero)

Ambas condiciones se cumplen.

9 es mayor a 0, y el residuo 0 es igual a 0

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 134

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

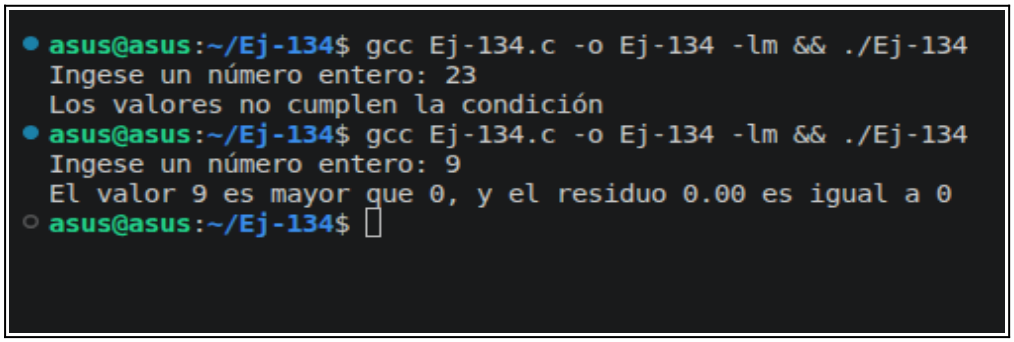


Fig.4 : Resultados de compilación del programa – 134.

- Enunciado del algoritmo

  142. Construya un algoritmo que lea el nombre de un estudiante (cadena) y tres notas (reales). Use una constante simbólica APROBADO = 7.0 como nota mínima de aprobación. Calcule el promedio con la expresión aritmética ((nota1 + nota2 + nota3) / 3). Evalúe la expresión relacional (promedio >= APROBADO). Muestre por pantalla un mensaje que incluya el nombre del estudiante, su promedio y su condición (aprobado o reprobado).

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Nombre del estudiante
  nota 1
  nota 2
  nota 3

- Procesos:

  ((nota1 + nota2 + nota3) / 3)

- Datos de salida:

  Promedio del estudiante
  Condición (aprobado, reprobado)

- Variables:

  nombre = “nombre ” ; →cadena
  nota 1 = 0.0 ; →real
  nota2 = 0.0 ; →real
  nota3 = 0.0 ; →real
  promedio = 0.0 ; →real

- Constantes:

  APROBADO = 7.0 ; →real

- Diagrama de flujo

  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

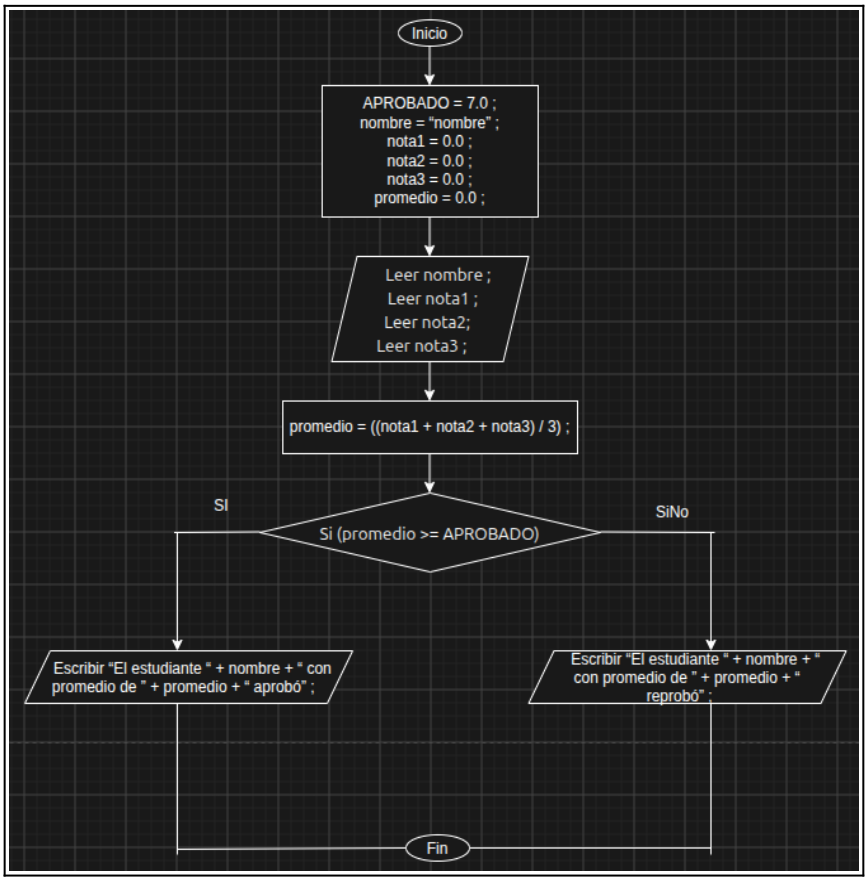


Fig.5: Diagrama de flujo en Draw.io – Algoritmo 142.

- Pseudocódigo

Inicio
APROBADO = 7.0 ;
nombre = "nombre" ;
nota1 = 0.0 ;
nota2 = 0.0 ;
nota3 = 0.0 ;
promedio = 0.0 ;
Leer nombre ;
Leer nota1 ;
Leer nota2;
Leer nota3 ;
promedio = ((nota1 + nota2 + nota3) / 3) ;
Si (promedio >= APROBADO)
Escribir "El estudiante " + nombre + " con promedio de " + promedio + " aprobó" ;
Sino
Escribir "El estudiante " + nombre + " con promedio de " + promedio + " reprobó" ;
FinSi ;
Fin

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Nombre del estudiante	Joel Paccha	---	---
Nota 1	9	---	---
Nota2	7.70	---	---
Nota 3	10	---	---
Promedio	---	((nota1 + nota2 + nota3) / 3)	8.90
Expresión Lógica	---	(promedio >= APROBADO)	La condición se cumple (Verdadero)

Resultado: 8.90 > 7.00.

El estudiante Joel Paccha, con promedio de 8.90, APROBÓ

Datos	Entradas	Procesos	Salidas
Nombre del estudiante	Joel Paccha	---	---
Nota 1	7.70	---	---
Nota2	10	---	---
Nota 3	2	---	---
Promedio	---	((nota1 + nota2 + nota3) / 3)	6.57
Expresión Lógica	---	(promedio >= APROBADO)	La condición no se cumple (Falso)

Resultado: 6.57 < 7.00.

El estudiante Joel Paccha, con promedio de 6.57, REPROBÓ

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 142

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

```
asus@asus:~/Ej-142$ gcc Ej-142.c -o Ej-142 -lm && ./Ej-142
Igrese el nombre: Joel Paccha
Igrese nota 1: 9
Igrese nota 2: 7.70
Igrese nota 3: 10
EL estudiante Joel Paccha, con promedio de 8.90 , APROBÓ
asus@asus:~/Ej-142$ gcc Ej-142.c -o Ej-142 -lm && ./Ej-142
Igrese el nombre: Joel Paccha
Igrese nota 1: 7.70
Igrese nota 2: 10
Igrese nota 3: 2
EL estudiante Joel Paccha, con promedio de 6.57 , REPROBÓ
asus@asus:~/Ej-142$
```

Fig.6: Resultados de compilación del programa – 142.

- Enunciado del algoritmo

  215. Declare una variable entera dato2 y asígnele el valor literal 100. Lea otra variable entera p por teclado. Calcule la expresión aritmética (dato2 * p + dato2 - p) y asígnela a una variable resultado. Luego evalúe la expresión relacional (resultado >= 0) y la expresión lógica (resultado > 100 OR dato2 > p). Muestre todos los valores por pantalla.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  P (Valor Ingresado)

- Procesos:

  (dato2 * p + dato2 - p)

- Datos de salida:

  Resultado de la operación
  Valor correcto e incorrecto
  Resultado mayor a 200 o valor ingresado menor a 100

- Variables:

  dato2 = 100 ; →entero
  p = 0 ; →real
  resultado = 0 ; →entero

- Constantes:

  N/A

- Diagrama de flujo

  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

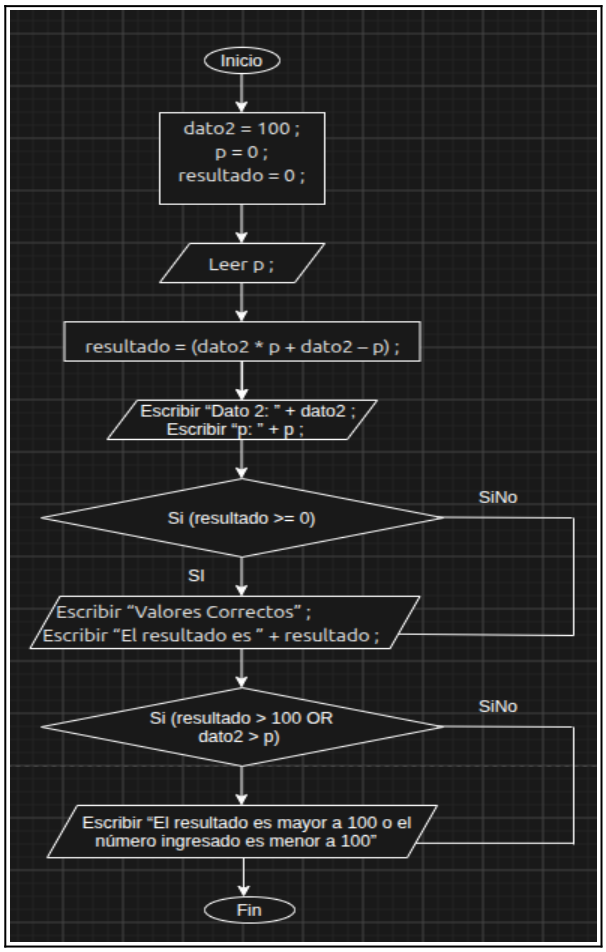


Fig.7: Diagrama de flujo en Draw.io – Algoritmo 215.

- Pseudocódigo

```
Inicio
dato2 = 100 ;
p = 0 ;
resultado = 0 ;
Leer p ;
resultado = (dato2 * p + dato2 – p) ;
Escribir “Dato 2: ” + dato2 ;
Escribir “p: ” + p ;
Si (resultado >= 0)
Escribir “Valores Correctos” ;
Escribir “El resultado es ” + resultado ;
FinSi
Si (resultado > 100 OR dato2 > p)
```

Escribir “El resultado es mayor a 100 o el número ingresado es menor a 100”
FinSi
Fin

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
p (Valor entero)	16	---	---
Dato 2	100 (asignado)	---	---
resultado	---	(dato2 * p + dato2 – p)	1684
Expresión Lógica	---	Si (resultado >= 0)	Se cumple (Verdadero)
Expresión Lógica	---	(resultado > 100 OR dato2 > p)	Ambas se cumplen (Verdadero)

Resultado: La primera condición se cumple (16 >= 0).

Valores correctos

Y la segunda condición también puesto que (1684 > 100) y (100 > 16).

El resultado es mayor a 100 o el número ingresado es menor a 100

Datos	Entradas	Procesos	Salidas
p (Valor entero)	321	---	---
Dato 2	100 (asignado)	---	---
resultado	---	(dato2 * p + dato2 – p)	31 879
Expresión Lógica	---	Si (resultado >= 0)	Se cumple (Verdadero)
Expresión Lógica	---	(resultado > 100 OR dato2 > p)	Se cumple una (Verdadero)

Resultado: La primera condición se cumple (321 >= 0).

Valores correctos

Y la segunda condición aunque la segunda expresión no se cumple (100 < 321), la primera expresión si se cumple (31 879 > 100) por lo que la condición es verdadera.

El resultado es mayor a 100 o el número ingresado es menor a 100

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 215

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

```
asus@asus:~/Ej-215$ gcc Ej-215.c -o Ej-215 -lm && ./Ej-215
Ingrese un número entero: 16
Dato 2: 100
Valor ingresado: 16
Valores Correctos
El resultado es: 1684
El resultado es mayor a 100 o el número ingresado es menor a 100
asus@asus:~/Ej-215$ gcc Ej-215.c -o Ej-215 -lm && ./Ej-215
Ingrese un número entero: 321
Dato 2: 100
Valor ingresado: 321
Valores Correctos
El resultado es: 31879
El resultado es mayor a 100 o el número ingresado es menor a 100
asus@asus:~/Ej-215$
```

Fig.8: Resultados de compilación del programa - 215

- Enunciado del algoritmo

  227. En un cine, cree un algoritmo que lea tres temperaturas (reales, en °C). Use la constante literal 3 como divisor. Calcule el promedio con la expresión aritmética ((t1 + t2 + t3) / 3) y asignelo a promedio. Evalúe la expresión lógica (promedio > -273.15). Muestre por pantalla la temperatura promedio. Ademas, determine si el resultado es par o impar (cuando aplique). Asegúrese de que las expresiones lógicas estén bien parentizadas.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Temperatura 1
  Temperatura 2
  Temperatura 3

- Procesos:

  ((t1 + t2 + t3) / 3)

- Datos de salida:

  Temperatura promedio
  Temperatura mayor, menor o igual a -273.15
  Si aplica par o impar
  Promedio par o impar

- Variables:

  t1 = 0.0 ; →real
  t2 = 0.0 ; →real
  t3 = 0.0 ; →real

promedio = 0.0 ; →real

- Constantes:
  constante = 3 ; →entero

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

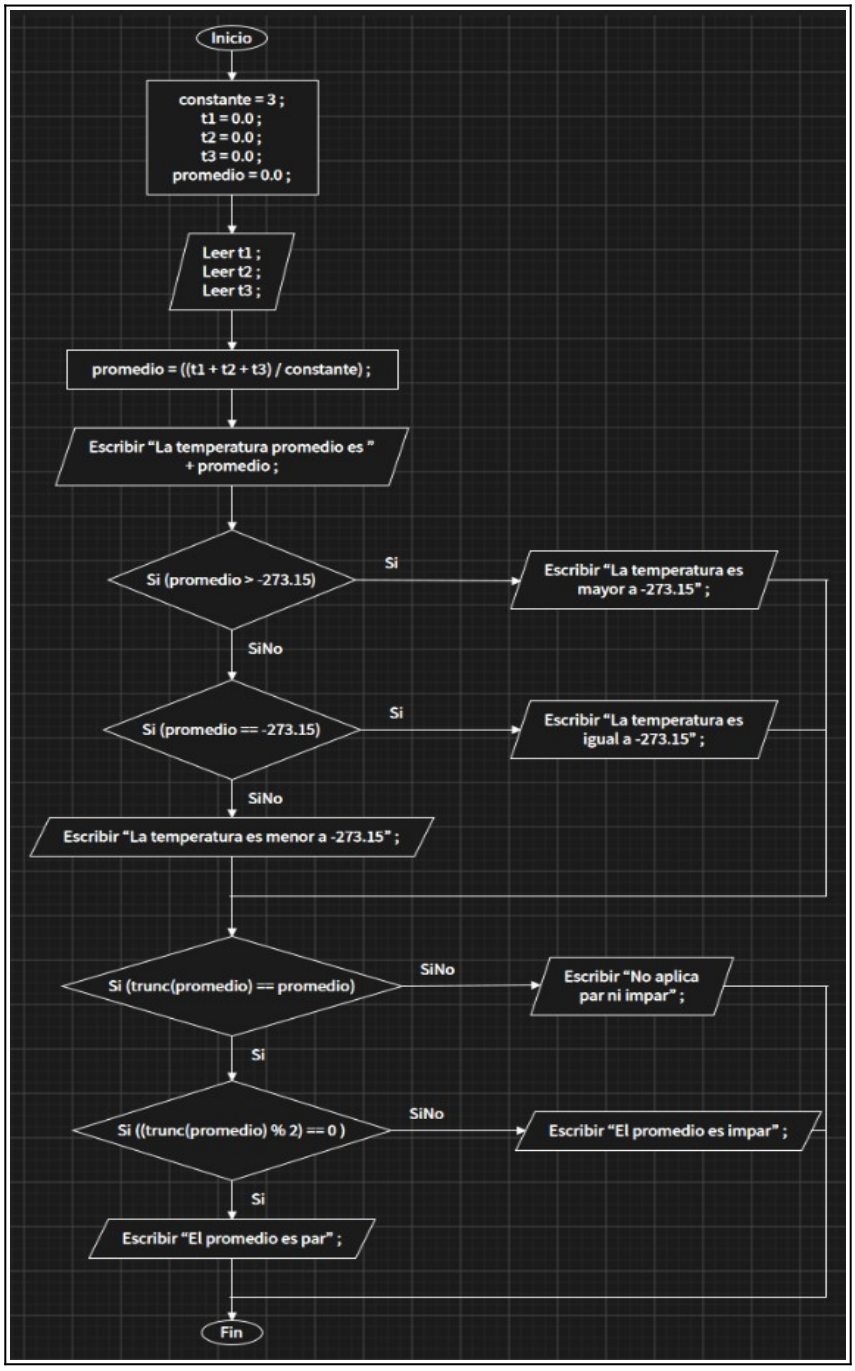


Fig.9: Diagrama de flujo en Draw.io – Algoritmo 227 .

- Pseudocódigo

```
Inicio
constante = 3 ;
t1 = 0.0 ;
t2 = 0.0 ;
t3 = 0.0 ;
promedio = 0.0 ;
Leer t1 ;
Leer t2 ;
Leer t3 ;
promedio = ((t1 + t2 + t3) / constante) ;
Escribir “La temperatura promedio es ” + promedio ;
Si (promedio > -273.15)
Escribir “La temperatura es mayor a -273.15” ;
Sino
Escribir “La temperatura es menor a -273.15” ;
Sino
Escribir “La temperatura es igual a -273.15” ;
FinSi ;
Si (trunc(promedio) == promedio)
    Si ((trunc(promedio) % 2)  == 0 )
    Escribir “El promedio es par” ;
    Sino
    Escribir “El promedio es impar” ;
    FinSi
Sino
Escribir “No aplica par ni impar” ;
FinSi
Fin
```

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Temperatura 1	28	---	---
Temperatura 2	31	---	---
Temperatura 3	40	---	---
promedio	---	((t1 + t2 + t3) / constante)	33
Expresión Lógica	---	Si (promedio > -273.15)	Se cumple (Verdadero)
Expresión Lógica	---	Si (trunc(promedio) == promedio)	Se cumple (Verdadero)
Expresión Lógica	---	Si ((trunc(promedio) %	No se cumple (Falso), es

		2) == 0)	impar

Resultado: El promedio de las temperaturas es 33, las primeras 2 condiciones se cumplen y la tercera condición no se cumple, por lo que el resultado es impar.

Temperatura promedio: 33 °C

Temperatura mayor a - 273.15

El promedio es impar

Datos	Entradas	Procesos	Salidas
Temperatura 1	26	---	---
Temperatura 2	10	---	---
Temperatura 3	24	---	---
promedio	---	((t1 + t2 + t3) / constante)	20
Expresión Lógica	---	Si (promedio > -273.15)	Se cumple (Verdadero)
Expresión Lógica	---	Si (trunc(promedio) == promedio)	Se cumple (Verdadero)
Expresión Lógica	---	Si ((trunc(promedio) % 2) == 0)	Se cumple (Verdadero), es par

Resultado: El promedio de las temperaturas es 20, todas las condiciones se cumplen.

Temperatura promedio: 20 °C

Temperatura mayor a - 273.15

El promedio es par

Datos	Entradas	Procesos	Salidas
Temperatura 1	15.28	---	---
Temperatura 2	34	---	---
Temperatura 3	19.76	---	---
promedio	---	((t1 + t2 + t3) / constante)	23.01
Expresión Lógica	---	Si (promedio > -273.15)	Se cumple (Verdadero).
Expresión Lógica	---	Si (trunc(promedio) == promedio)	No se cumple (Falso), no aplica par ni impar
Expresión Lógica	---	Si ((trunc(promedio) % 2) == 0)	No aplica.

Resultado: El promedio de las temperaturas es 23,01, la primera condición se cumple, pero las otras no, ya que el promedio es decimal, por lo que no aplica par ni impar.

Temperatura promedio: 23.01 °C

Temperatura mayor a - 273.15

No aplica par ni impar puesto que es decimal

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 227

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

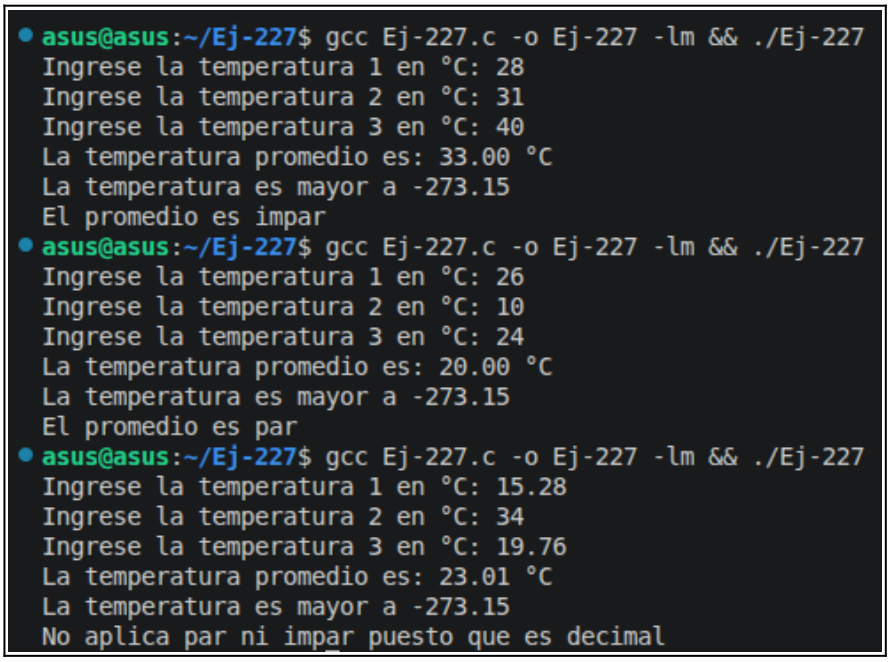


Fig.10: Resultados de compilación del programa – 227.

- Enunciado del algoritmo

  229. Cree un algoritmo que declare las variables num2 (entero), dato2 (real) y num1 (carácter). Lea tres valores por teclado en el orden: entero, real y carácter. Evalúe la expresión lógica (num2 > 0 AND dato2 > 0.0 AND num1 != ' '). Muestre por pantalla un mensaje indicando si todos los valores son válidos (positivos y no vacíos). Identifique el tipo de dato simple de cada variable.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Valor entero

  Valor real

Valor carácter

- Procesos:
  Expresión lógica: (num2 > 0 AND dato2 > 0.0 AND num1 != ' ')

- Datos de salida:
  Valores ingresados válidos o inválidos
  Razón de valores inválidos (negativos o vacíos)

- Variables:
  num2 = 0. ; →entero
  dato2 = 0.0 ; →real
  num1 ; →carácter

- Constantes:
  N/A

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

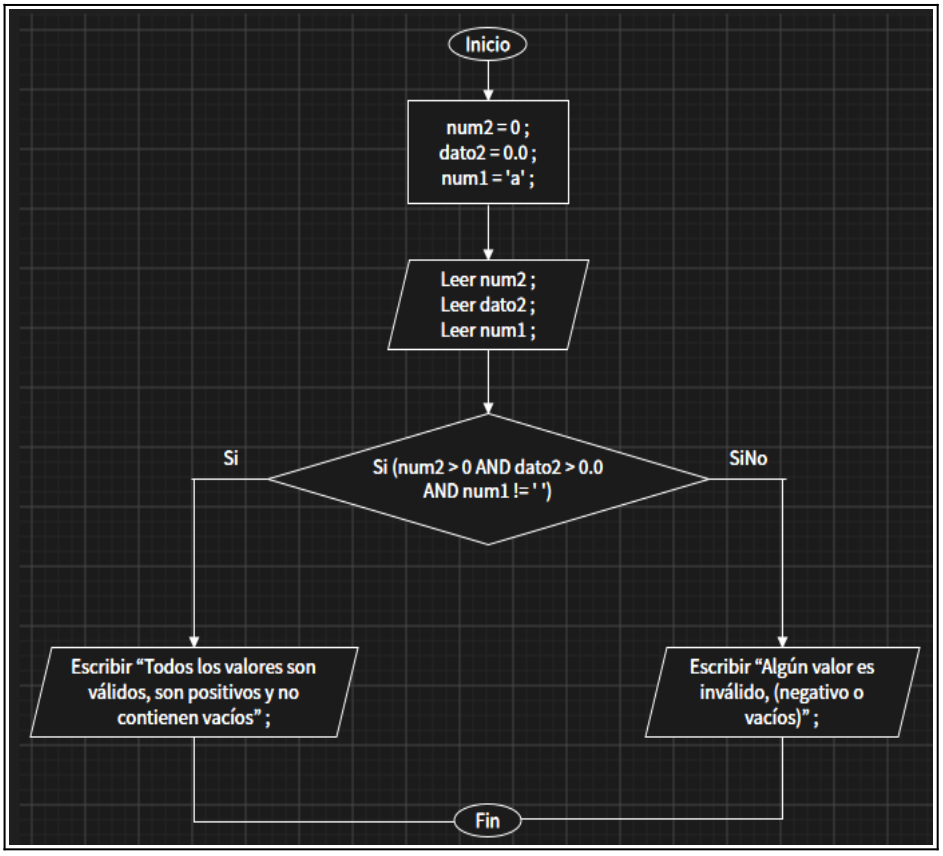


Fig.11: Diagrama de flujo en Draw.io – Algoritmo 229 .

- Pseudocódigo

  Inicio
  num2 = 0 ;
  dato2 = 0.0 ;
  num1 = 'a' ;
  Leer num2 ;
  Leer dato2 ;
  Leer num1 ;
  Si (num2 > 0 AND dato2 > 0.0 AND num1 != ' ') ;
  Escribir “Todos los valores son válidos, son positivos y no contienen vacíos” ;
  SiNo ;
  Escribir “Algún valor es inválido, (negativo o vacíos)” ;
  FinSi ;
  Fin

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Número entero (num2)	5	---	---
Número real (dato2)	5.5	---	---
Carácter (dato1)		---	---
Expresión Lógica	---	Si (num2 > 0 AND dato2 > 0.0 AND num1 != ' ')	Se cumplen las dos primeras expresiones de la condición (num2 real, y dato2 entero), pero la tercera expresión no se cumple ya que num1 presenta un espacio vacío, por lo cual el resultado es falso

Resultado: La primera y segunda expresión de la condición se cumplen, pero la tercera no.
Algún valor es inválido, (negativo o vacíos)

Datos	Entradas	Procesos	Salidas
Número entero (num2)	4	---	---
Número real (dato2)	7.9	---	---
Carácter (dato1)	w	---	---
Expresión Lógica	---	Si (num2 > 0 AND dato2 > 0.0 AND num1 != ' ')	Se cumplen todas las expresiones de la condición (num2 real, dato2 entero, y dato1 carácter), por lo cual es resultado es verdadero

Resultado: Todas las expresiones de la condición se cumplen.
Todos los valores son válidos, son positivos y no contienen vacíos.

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 229

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

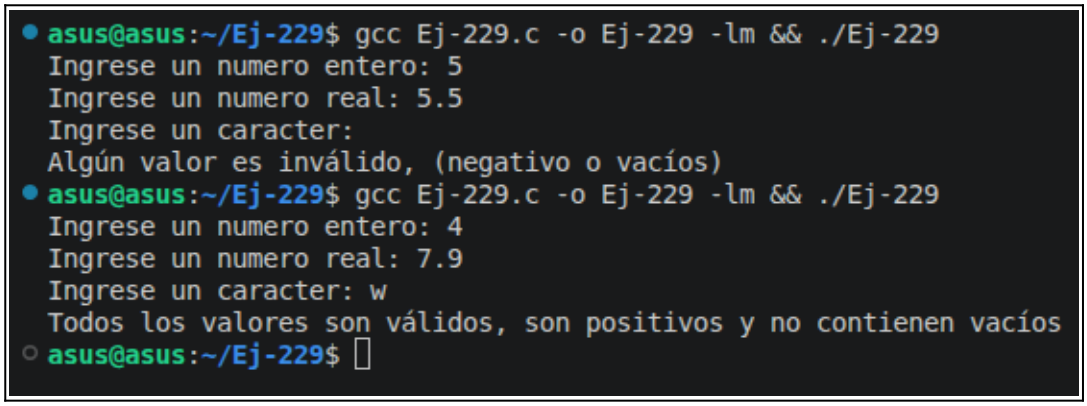


Fig.12: Resultados de compilación del programa – 229.

- Enunciado del algoritmo

  251. En un hospital, diseñe un algoritmo que evalúe la calidad del aire. Lea los niveles de PM2.5 (real), PM10 (real), O3 (real), NO2 (real) y SO2 (real). Use constantes simbólicas LIMITE_PM25 = 25, LIMITE_PM10 = 50, LIMITE_O3 = 100, LIMITE_NO2 = 200, LIMITE_SO2 = 125. Evalúe las expresiones lógicas relacionales para cada parámetro. Calcule el indice de calidad del aire con la expresión aritmética máx de los cocientes. Asigne la categoría a una cadena. Muestre por pantalla la calidad del aire. Ademas del calculo principal, calcule el resto de una división adicional. Agregue al final un resumen de los datos ingresados.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Niveles: PM2.5
  PM10
  O3
  NO2
  SO2

- Procesos:
  PM25/ LIMITE_PM25
  PM10/ LIMITE_PM10
  O3/ LIMITE_O3
  NO2/ LIMITE_NO2
  SO2/ LIMITE_SO2
  Expresiones lógicas: (indicePM25> indice)
  (indicePM10 > indice)
  Si (indiceO3 > indice)
  (indiceNO2 > indice)
  (indiceSO2> indice)
  (PM25 > LIMITE_PM25 OR PM10 > LIMITE_PM10 OR O3 > LIMITE_O3 OR NO2
  LIMITE_NO2 OR SO2 > LIMITE_SO2)
  trunc(PM25) % 2

- Datos de salida:
  Índice
  Calidad
  Resto de una división adicional
  Resumen de los valores ingresados.

- Variables:
  PM25 = 0.0 →real
  PM10 = 0.0 →real
  O3 = 0.0 →real
  NO2 = 0.0 →real
  SO2 = 0.0 →real
  indicePM25 = 0.0 →real
  indicePM10 = 0.0 →real
  indiceO3 = 0.0 →real
  indiceNO2 = 0.0 →real
  indiceSO2 = 0.0 →real
  indice = 0.0 →real
  resto = 0.0 →real
  calidad = “” →cadena

- Constantes:
  LIMITE_PM25 = 25 →entero
  LIMITE_PM10 = 50 →entero
  LIMITE_O3 = 100 →entero
  LIMITE_NO2 = 200 →entero
  LIMITE_SO2 = 125 →entero

- Diagrama de flujo

  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

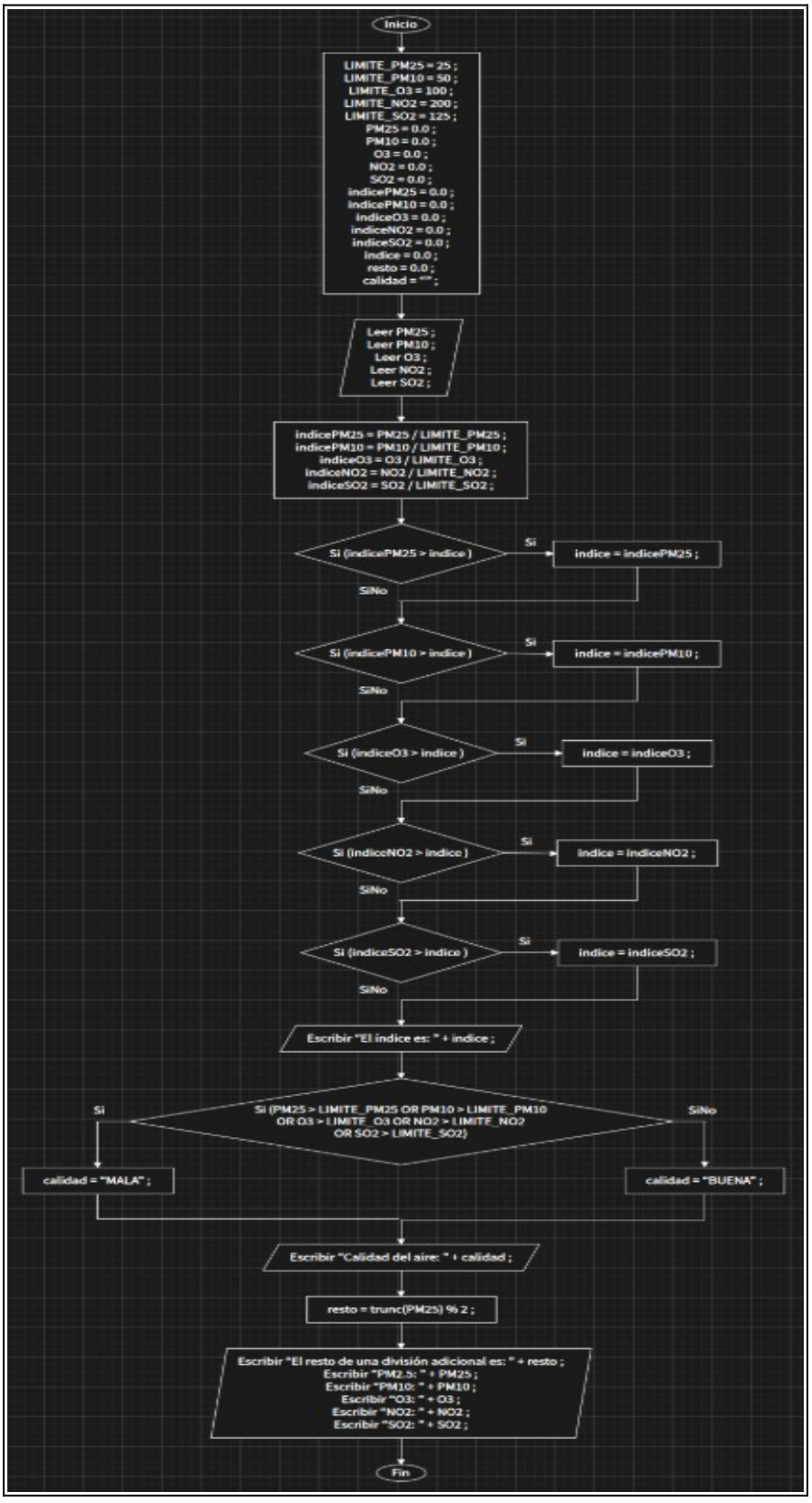


Fig.13: Diagrama de flujo en Draw.io – Algoritmo 251 .

- Pseudocódigo

```
Inicio
LIMITE_PM25 = 25 ;
LIMITE_PM10 = 50 ;
LIMITE_O3 = 100 ;
LIMITE_NO2 = 200 ;
LIMITE_SO2 = 125 ;
PM25 = 0.0 ;
PM10 = 0.0 ;
O3 = 0.0 ;
NO2 = 0.0 ;
SO2 = 0.0 ;
indicePM25 = 0.0 ;
indicePM10 = 0.0 ;
indiceO3 = 0.0 ;
indiceNO2 = 0.0 ;
indiceSO2 = 0.0 ;
indice = 0.0 ;
resto = 0.0 ;
calidad = “” ;
Leer PM25 ;
Leer PM10 ;
Leer O3 ;
Leer NO2 ;
Leer SO2 ;
indicePM25 = PM25/ LIMITE_PM25 ;
indicePM10 = PM10/ LIMITE_PM10 ;
indiceO3 = O3/ LIMITE_O3 ;
indiceNO2 = NO2/ LIMITE_NO2 ;
indiceSO2 = SO2/ LIMITE_SO2 ;
Si (indicePM25> indice )
indice = indicePM25 ;
FinSi
Si (indicePM10 > indice )
indice = indicePM10 ;
FinSi
Si (indiceO3 > indice )
indice = indiceO3 ;
FinSi
Si (indiceNO2 > indice )
indice = indiceNO2 ;
FinSi
Si (indiceSO2> indice )
```

```
indice = indiceSO2  ;
FinSi
Escribir “El índice es:  ” + indice ;
Si (PM25 > LIMITE_PM25 OR  PM10 > LIMITE_PM10 OR O3 > LIMITE_O3 OR NO2
> LIMITE_NO2 OR SO2 > LIMITE_SO2)
calidad = “MALA” ;
Sino
calidad = “BUENA” ;
FinSi
Escribir “Calidad del aire: ” + calidad ;
resto = trunc(PM25) % 2 ;
Escribir “EL resto de una división adicional es: ” + resto ;
Escribir “PM2.5: ” + PM25 ;
Escribir “PM10: ” + PM10 ;
Escribir “O3: ” + O3 ;
Escribir “NO2: ” + NO2 ;
Escribir “SO2: ” + SO2 ;
Fin
```

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
PM25	20	---	---
PM10	43	---	---
O3	52	---	---
NO2	98	---	---
SO2	100.98	---	---
indicePM25	---	PM25/ LIMITE_PM25	0.8
indicePM10	---	PM10/ LIMITE_PM10	0.86
indiceO3	---	O3/ LIMITE_O3	0.52
indiceNO2	---	NO2/ LIMITE_NO2	0.49
indiceSO2	---	SO2/ LIMITE_SO2	0.80784
Expresión Lógica	---	Si (indicePM25> indice)	Falso
Expresión Lógica	---	Si (indicePM10 > indice)	Verdadero
Expresión Lógica	---	Si (indiceO3 > indice)	Falso
Expresión Lógica	---	Si (indiceNO2 > indice)	Falso
Expresión Lógica	---	Si (indiceSO2> indice)	Falso
indice		---	0.86
Expresión Lógica	---	Si (PM25 > LIMITE_PM25 OR PM10 > LIMITE_PM10 OR O3 > LIMITE_O3	No se cumple (Falso),

		OR NO2 > LIMITE_NO2 OR SO2 > LIMITE_SO2)	por lo tanto la calidad del aire es BUENA
resto	---	trunc(PM25) % 2	0

Resultado:
Índice: 0.86
Calidad del aire: BUENA
Resto de una división adicional: 0

Temperatura promedio: 33 °C
Temperatura mayor a - 273.15
El promedio es impar

Datos	Entradas	Procesos	Salidas
PM25	32	---	---
PM10	35.98	---	---
O3	97.06	---	---
NO2	187	---	---
SO2	147.89	---	---
indicePM25	---	PM25/ LIMITE_PM25	1.28
indicePM10	---	PM10/ LIMITE_PM10	0.7196
indiceO3	---	O3/ LIMITE_O3	0.9706
indiceNO2	---	NO2/ LIMITE_NO2	0.935
indiceSO2	---	SO2/ LIMITE_SO2	1.18312
Expresión Lógica	---	Si (indicePM25> indice)	Verdadero
Expresión Lógica	---	Si (indicePM10 > indice)	Falso
Expresión Lógica	---	Si (indiceO3 > indice)	Falso
Expresión Lógica	---	Si (indiceNO2 > indice)	Falso
Expresión Lógica	---	Si (indiceSO2> indice)	Falso
indice		---	1.28
Expresión Lógica	---	Si (PM25 > LIMITE_PM25 OR PM10 > LIMITE_PM10 OR O3 > LIMITE_O3 OR NO2 > LIMITE_NO2 OR SO2 > LIMITE_SO2)	Se cumple (Verdadero), por lo tanto la calidad del aire es MALA
resto	---	trunc(PM25) % 2	0

Resultado:
Índice: 1.28
Calidad del aire: MALA
Resto de una división adicional: 0

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 251

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

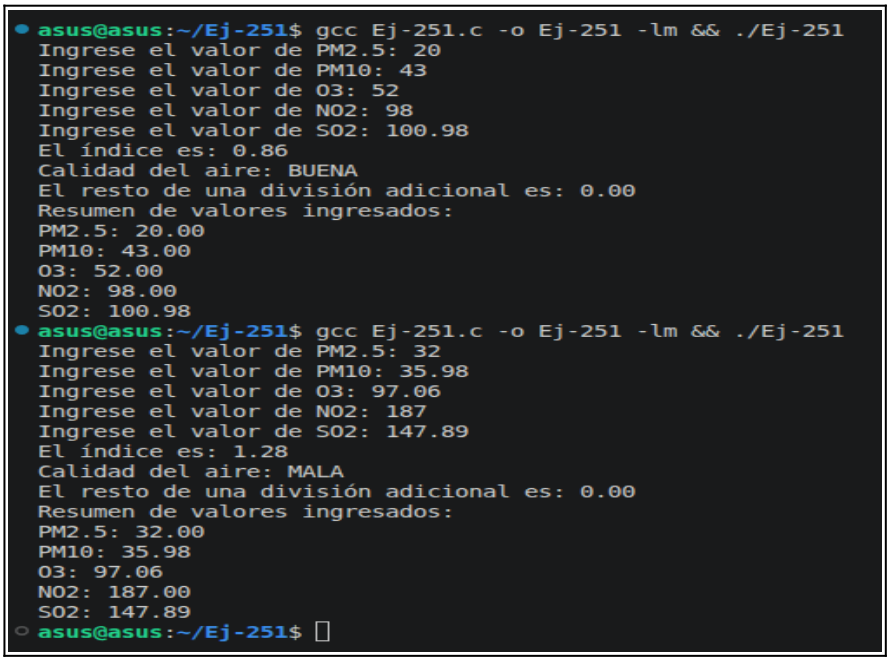


Fig.14: Resultados de compilación del programa – 251.

- Enunciado del algoritmo

  252 Diseñe un algoritmo que valide una dirección de correo electrónico ingresada por teclado (cadena). Use constantes simbólicas MIN_LEN = 5, MAX_LEN = 50. Evalúe la expresión lógica compuesta: (longitud(correo) >= MIN_LEN AND longitud(correo) <= MAX_LEN AND contiene(carreo, '@') AND contiene(carreo, '.') AND NOT empieza_con(carreo, '@') AND NOT termina_con(carreo, '@')). Asigne el resultado a una variable booleana correo_valido. Muestre por pantalla si el correo es válido o no, indicando qué criterios fallaron. Identifique dato vs información.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:
  Dirección de correo electrónico

- Procesos:
  (longitud(correo) >= MIN_LEN AND longitud(correo) <= MAX_LEN AND contiene(carreo, '@') AND contiene(carreo, '.') AND NOT empieza_con(carreo, '@') AND NOT termina_con(carreo, '@'))

- Datos de salida:
  Correo válido o inválido
  Razón de invalidez: Caracteres insuficientes
  Demasiados caracteres
  No contiene ‘@’
  No contiene ‘.’
  Empieza con ‘@’
  Termina con ‘@’

- Variables:
  correo = “correo” →cadena

- Constantes:
  correo_valido = false →booleano
  MIN_LEN = 5 →entero
  MAX_LEN = 5O →entero

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

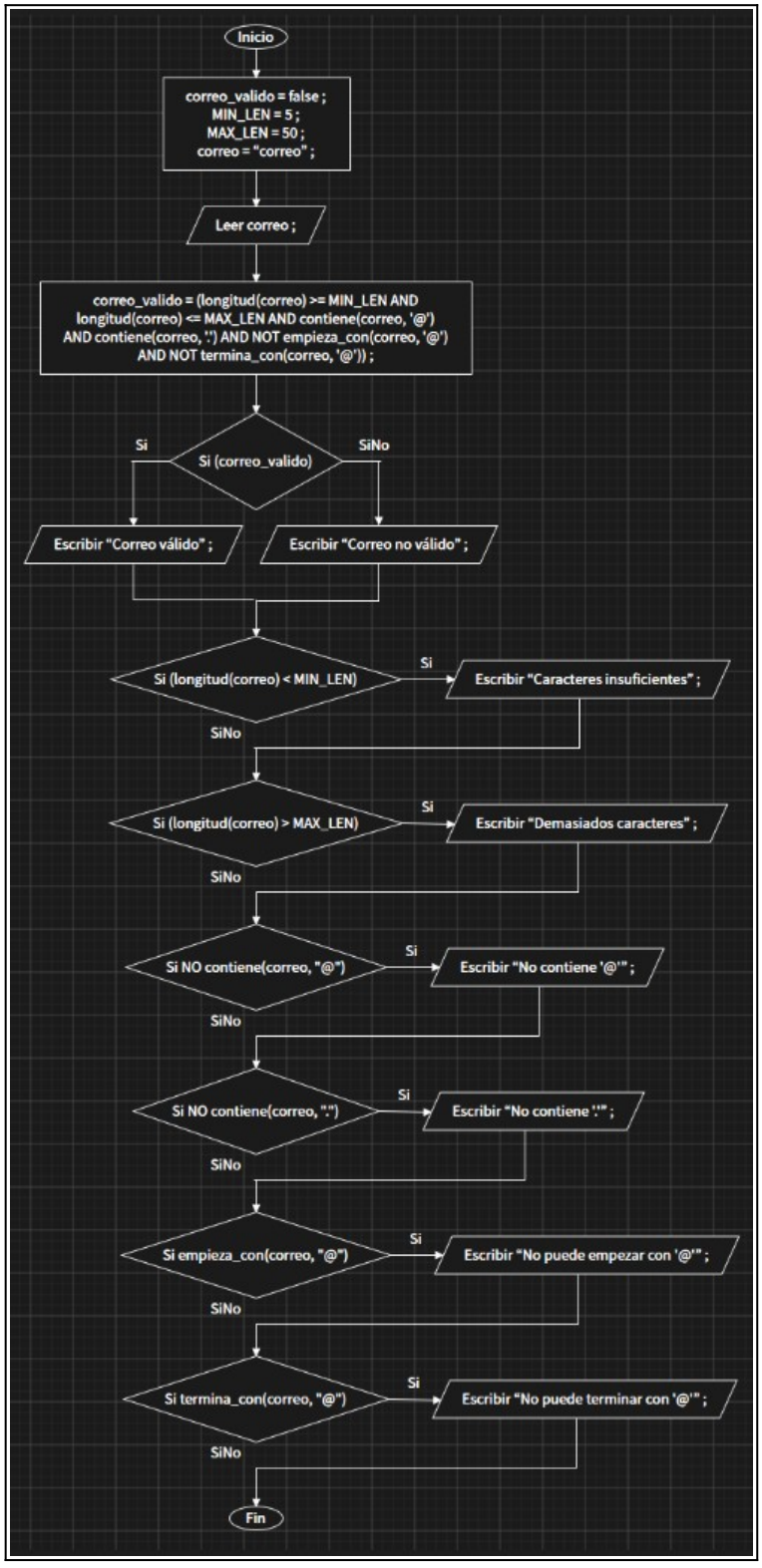


Fig.15: Diagrama de flujo en Draw.io – Algoritmo 252 .

- Pseudocódigo

  Inicio

  correo_valido = false ;

```
MIN_LEN = 5 ;
MAX_LEN = 50 ;
correo = “correo” ;
Leer correo ;
correo_valido = (longitud(correo) >= MIN_LEN AND longitud(correo) <= MAX_LEN
AND contiene(correo, '@') AND contiene(correo, '.') AND NOT empieza_con(correo, '@')
AND NOT termina_con(correo, '@')) ;
Si (correo_valido)
Escribir “Correo válido” ;
Sino
Escribir “Correo no válido”  ;
FinSi
Si (longitud(correo) < MIN_LEN)
Escribir “Caracteres insuficientes” ;
FinSi
Si (longitud(correo) > MAX_LEN)
Escribir “Demasiados caracteres” ;
FinSi
Si NO contiene(correo, "@")
Escribir " No contiene '@' " ;
FinSi
Si NO contiene(correo, ".")
Escribir "No contiene '.' " ;
FinSi
Si empieza_con(correo, "@")
Escribir "No puede empezar con '@' " ;
FinSi
Si termina_con(correo, "@")
Escribir " No puede terminar con '@' " ;
FinSi
Fin
```

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
correo	joelpaccha@gmail.com	---	---
correo_valido	---	correo_valido = (longitud(correo) >= MIN_LEN AND longitud(correo) <= MAX_LEN AND contiene(correo, '@') AND contiene(correo, '.') AND NOT empieza_con(correo,	Se cumple (Verdadero), por lo tanto el correo es válido.

		'@') AND NOT termina_con(correo, '@'))	

Resultado:

Correo válido

Datos	Entradas	Procesos	Salidas
correo	@paccha2008@	---	---
correo_valido	---	correo_valido = (longitud(correo) >= MIN_LEN AND longitud(correo) <= MAX_LEN AND contiene(correo, '@') AND contiene(correo, '.') AND NOT empieza_con(correo, '@') AND NOT termina_con(correo, '@'))	La condición no se cumple (Falso), por lo cual el correo es invalido.
Expresión lógica	---	Si (longitud(correo) < MIN_LEN)	---
Expresión lógica	---	Si (longitud(correo) > MAX_LEN)	---
Expresión lógica	---	Si NO contiene(correo, "@")	---
Expresión lógica	---	Si NO contiene(correo, ".")	No contiene '.'
Expresión lógica	---	Si empieza_con(correo, "@")	Si empieza con '@'
Expresión lógica	---	Si termina_con(correo, "@")	Si termina con '@'

Resultado:

Correo no válido

No contiene punto (.)

No puede empezar con '@'

No puede terminar con '@'

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 252

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

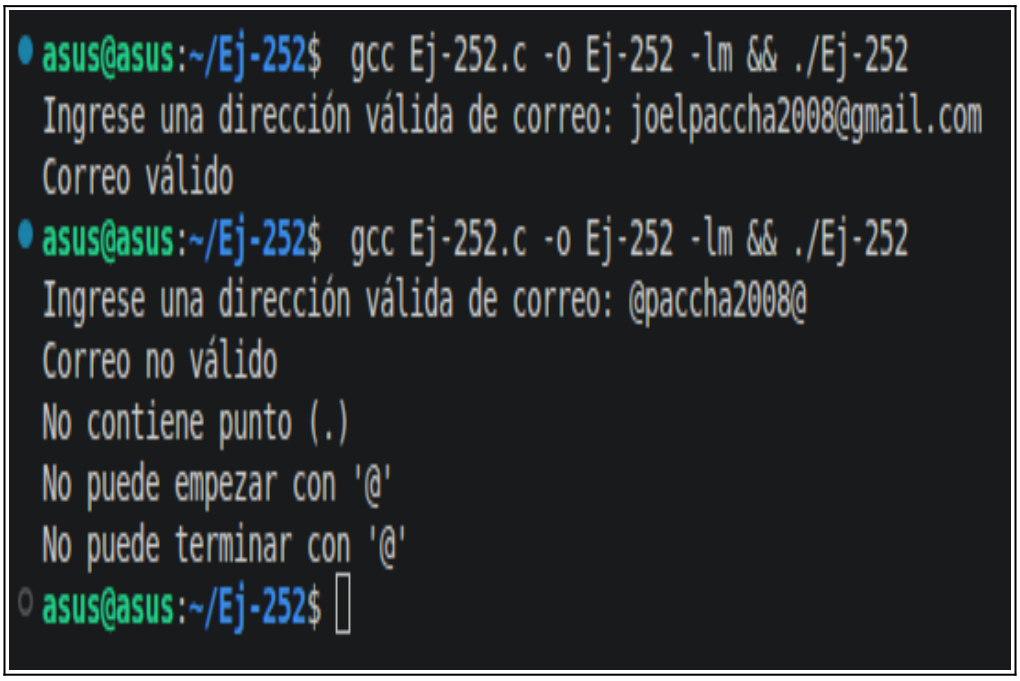


Fig.16: Resultados de compilación del programa – 252.

- Enunciado del algoritmo

  253. En una farmacia, cree un algoritmo que controle las ventas mensuales. Lea las ventas del mes (real), la meta mensual (real), el porcentaje de comisión (real), el numero de clientes nuevos (entero) y el numero de devoluciones (entero). Use constantes simbólicas BONO_META = 0.05, DESC_DEVOLUCIONES = 0.02, MAX_DEVOLUCIONES = 5. Evalúe la expresión lógica (ventas >= meta). Calcule la comisión con expresiones aritméticas. Muestre por pantalla el detalle. Ademas del calculo principal, calcule el resto de una división adicional. Identifique explícitamente los datos de entrada y la información de salida.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Ventas del mes
  Meta mensual
  Porcentaje de comisión
  Número de clientes nuevos

Número de devoluciones

- Procesos:
  (ventas * porcentaje) / 100
  comisiones *BONO_META
  Si (devoluciones > MAX_DEVOLUCIONES)
  descuento = (comision_total * DESC_DEVOLUCIONES) * devoluciones ;
  descuento_total = comision_total – descuento

- Datos de salida:
  Comisión total
  Descuento por devoluciones
  Resto de una división adicional

- Variables:
  ventas = 0.0 →real
  meta = 0.0 →real
  porcentaje = 0.0 →real
  clientes_nuevos = 0 →real
  devoluciones = 0 →real
  bono = 0.0 →real
  comisiones = 0.0 →real
  comision_total = 0.0 →real
  descuento = 0.0 →real
  descuento_total = 0.0 →real
  resto = 0 →entero

- Constantes:
  BONO_META = 0.05 →real
  DESC_DEVOLUCIONES = 0.02 →real
  MAX_DEVOLUCIONES = 5 →entero

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

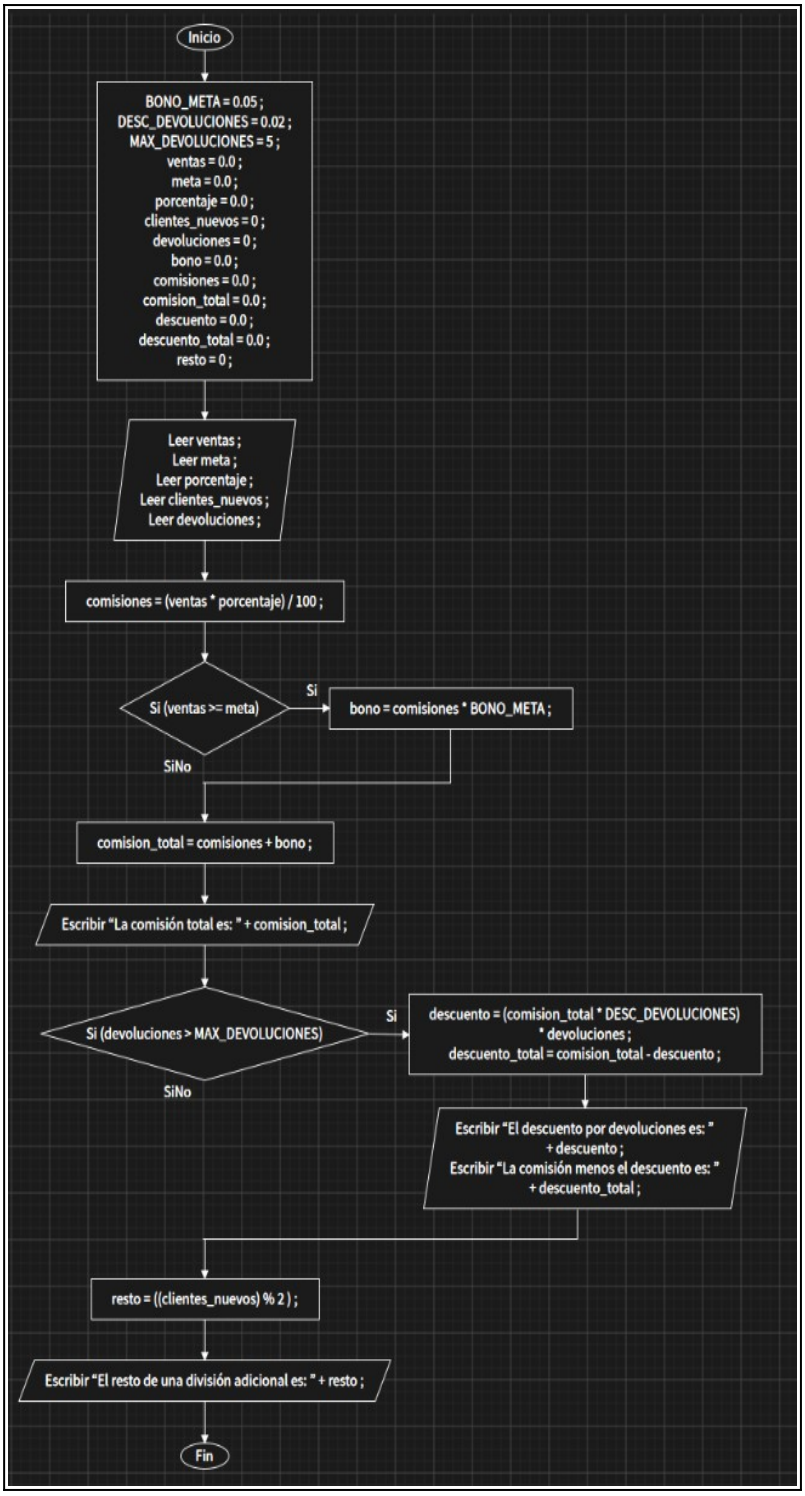


Fig.17: Diagrama de flujo en Draw.io – Algoritmo 253 .

- Pseudocódigo

  Inicio
  BONO_META = 0.05 ;
  DESC_DEVOLUCIONES = 0.02 ;
  MAX_DEVOLUCIONES = 5 ;
  ventas = 0.0 ;

```
meta = 0.0 ;
porcentaje = 0.0 ;
clientes_nuevos = 0 ;
devoluciones = 0 ;
bono = 0.0 ;
comisiones = 0.0 ;
comision_total = 0.0 ;
descuento = 0.0 ;
descuento_total = 0.0  ;
resto = 0  ;
Leer ventas ;
Leer meta ;
Leer porcentaje ;
Leer clientes_nuevos ;
Leer devoluciones ;
comisiones = (ventas * porcentaje) / 100 ;
Si (ventas >= meta)
bono = comisiones *BONO_META;
FinSi
comision_total = comisiones + bono ;
Escribir “La comisión total es: ” + comision_total ;
Si (devoluciones > MAX_DEVOLUCIONES)
descuento = (comision_total * DESC_DEVOLUCIONES) * devoluciones ;
descuento_total = comision_total - descuento ;
Escribir “El descuento por devoluciones es: ” + descuento ;
Escribir “La comisión menos el descuento es: ” + descuento_total ;
FinSi
resto = ((clientes_nuevos) % 2 ) ;
Escribir “El resto de una división adicional es: ” + resto ;
Fin
```

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Ventas	147	---	---
Meta mensual	100	---	
Porcentaje de comisión	6	---	---
Clientes nuevos	59	---	---
Devoluciones	4	---	---
comisiones	---	(ventas * porcentaje) / 100	8.82
Expresión lógica	---	(ventas >= meta)	Se cumple (Verdadero)
bono	---	comisiones	0.441

		*BONO_META	
comisión total	---	comisiones + bono	9.26
Expresión lógica	---	Si (devoluciones > MAX_DEVOLUCIONES)	No se cumple (Falso)
descuento	---	(comision_total * DESC_DEVOLUCIONES) * devoluciones	No aplica
Descuento total	---	comision_total - descuento	No aplica
resto	---	((clientes_nuevos) % 2)	1

Resultado:

La comisión total es 9.26

El resto de una división adicional es 1

Datos	**Entradas**	**Procesos**	**Salidas**
Ventas	86	---	---
Meta mensual	100	---	
Porcentaje de comisión	3	---	---
Clientes nuevos	78	---	---
Devoluciones	9	---	---
comisiones	---	(ventas * porcentaje) / 100	2.58
Expresión lógica	---	(ventas >= meta)	No se cumple (Falso)
bono	---	comisiones *BONO_META	No aplica
comisión total	---	comisiones + bono	2.58
Expresión lógica	---	Si (devoluciones > MAX_DEVOLUCIONES)	Se cumple (Verdadero)
descuento	---	(comision_total * DESC_DEVOLUCIONES) * devoluciones	0.46
Comisión – Descuento total	---	comision_total - descuento	2.12
resto	---	((clientes_nuevos) % 2)	0

Resultado:

La comisión total es: 2.58

El descuento por devoluciones es: 0.46

La comisión menos el descuento es 2.12
El resto de una división adicional es: 0

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.
  Ejercicio – 253

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

```
asus@asus:~/Ej-253$ gcc Ej-253.c -o Ej-253 -lm && ./Ej-253
Ingrese el número de ventas: 147
Ingrese la meta mensual: 100
Ingrese el porcentaje de comisión: 6
Ingrese el número de clientes nuevos: 59
Ingrese el número de devoluciones: 4
La comision total es: 9.26
El resto de una división adicional es: 1
asus@asus:~/Ej-253$ gcc Ej-253.c -o Ej-253 -lm && ./Ej-253
Ingrese el número de ventas: 86
Ingrese la meta mensual: 100
Ingrese el porcentaje de comisión: 3
Ingrese el número de clientes nuevos: 78
Ingrese el número de devoluciones: 9
La comision total es: 2.58
El descuento por devoluciones es: 0.46
La comision menos el descuento es: 2.12
El resto de una división adicional es: 0
asus@asus:~/Ej-253$
```

Fig.18: Resultados de compilación del programa – 253.

- Enunciado del algoritmo

  254. En un taller mecánico, diseñe un algoritmo que calcule el costo de una mudanza. Lea la distancia (real, km), el peso total (real, kg), el volumen (real, m3), el numero de habitaciones (entero) y el servicio adicional (booleano). Use constantes simbólicas TARIFA_BASE = 100.0, COSTO_KM = 1.0, COSTO_KG = 0.5, COSTO_M3 = 30.0, COSTO_HABITACION = 25.0, COSTO_EMPAQUE = 200.0. Calcule el costo con expresiones aritmeticas. Muestre por pantalla el desglose. Ademas del calculo principal, calcule el resto de una división adicional. Asegúrese de inicializar todas las variables antes de usarlas.

- Análisis del problema
  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:
  Distancia
  Peso total
  Volumen
  Número habitaciones
  Servicio adicional

- Procesos:
  TARIFA_BASE + (distanciakm * COSTO_KM) + (peso_totalkg * COSTO KG) + (volumen_m3 * COSTO_M3) + (numerohabitaciones * COSTO_HABITACION)
  costo_final: costo_total + COSTO_EMPAQUE
  resto = (numerohabitaciones) % 2)

- Datos de salida:
  Costo total
  Resumen de datos ingresados

- Variables:
  distanciakm = 0.0 →real
  peso_totalkg = 0.0 →real
  volumen_m3 = 0.0 →real
  numerohabitaciones = 0 →entero
  respuesta = “” →carácter
  servicio_adicional = false →booleano
  costo_total = 0.0 →real
  costo_final = 0.0 →real
  resto = 0 →entero

- Constantes:
  TARIFA_BASE = 100 →entero
  COSTO_KM = 1.0 →real
  COSTO_KG = 0.5 →real
  COSTO_M3 = 30.0 →real
  COSTO_HABITACION = 25.0 →real
  COSTO_EMPAQUE = 200.0 →real

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

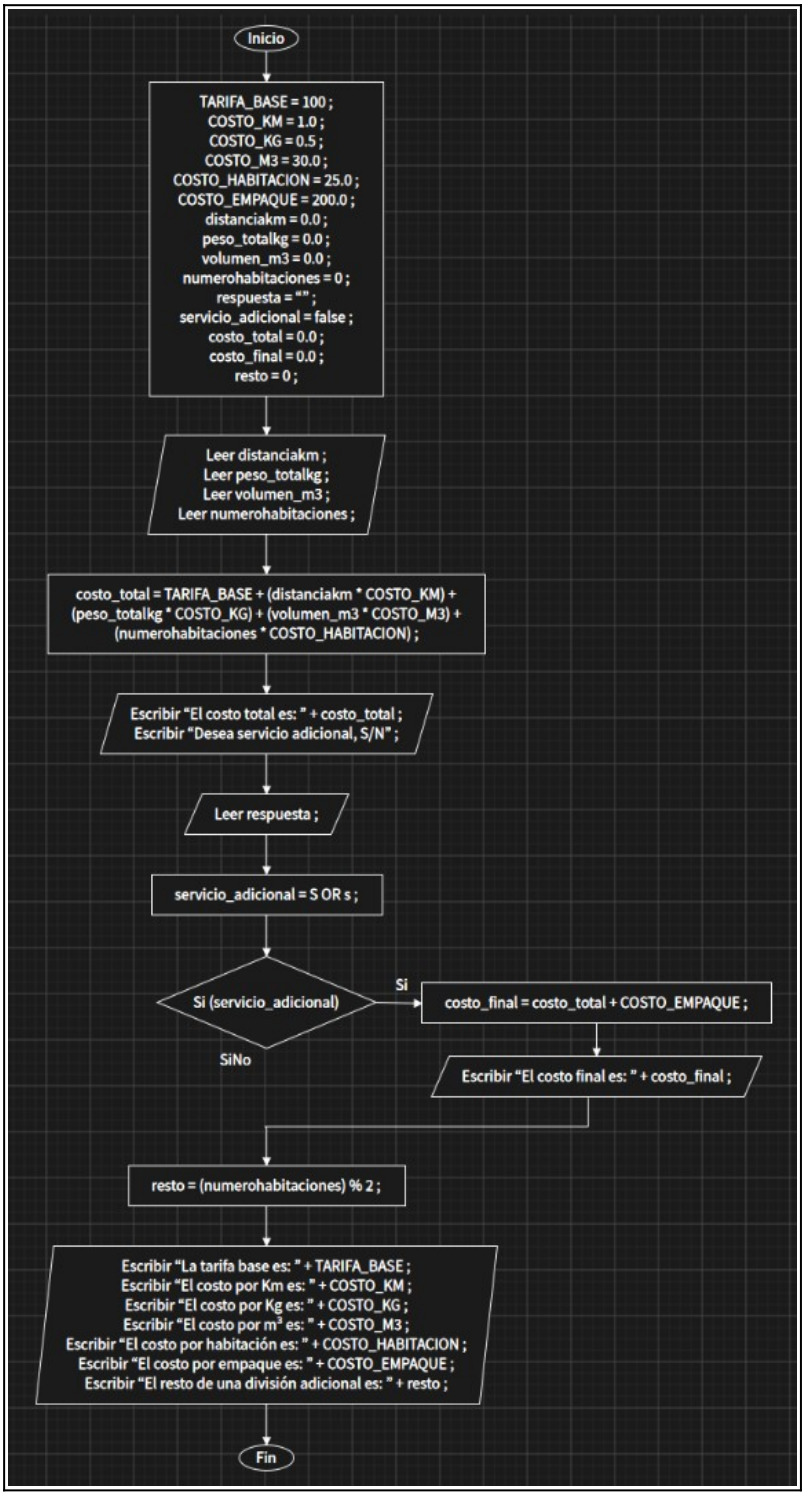


Fig.19: Diagrama de flujo en Draw.io – Algoritmo 254 .

- Pseudocódigo

  Inicio
  TARIFA_BASE = 100 ;
  COSTO_KM = 1.0 ;
  COSTO_KG = 0.5 ;

COSTO_M3 = 30.0 ;
COSTO_HABITACION = 25.0 ;
COSTO_EMPAQUE = 200.0 ;
distanciakm = 0.0 ;
peso_totalkg = 0.0 ;
volumen_m3 = 0.0 ;
numerohabitaciones = 0 ;
respuesta = “” ;
servicio_adicional = false ;
costo_total = 0.0 ;
costo_final = 0.0 ;
resto = 0 ;
Leer distanciakm ;
Leer peso_totalkg ;
Leer volumen_m3 ;
Leer numerohabitaciones ;
costo_total = TARIFA_BASE + (distanciakm * COSTO_KM) + (peso_totalkg * COSTO_KG) + (volumen_m3 * COSTO_M3) + (numerohabitaciones * COSTO_HABITACION) ;
Escribir “El costo total es: ” + costo_total ;
Escribir “Desea servicio adicional, S/N” ;
Leer respuesta ;
servicio_adicional = S OR s ;
Si (servicio_adicional)
costo_final= costo_total + COSTO_EMPAQUE ;
Escribir “El costo final es: ” + costo_final ;
FinSi
resto = (numerohabitaciones) % 2 ;
Escribir “La tarifa base es: ” + TARIFA_BASE ;
Escribir “El costo por Km es: ” + COSTO_KM ;
Escribir “El costo por Kg es: ” + COSTO_KG ;
Escribir “EL costo por m^3 es: ” + COSTO_M3 ;
Escribir “El costo por habitación es: ” + COSTO_HABITACION ;
Escribir “El costo por empaque es: ” + COSTO_EMPAQUE ;
Escribir “El resto de una división adicional es: ” + resto ;
Fin

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Distancia	11	---	---
Peso	530.89	---	
Volumen	4.3	---	---
Número habitaciones	3	---	---

Costo total	---	TARIFA_BASE + (distanciakm * COSTO_KM) + (peso_totalkg * COSTO_KG) + (volumen_m3 * COSTO_M3) + (numerohabitaciones * COSTO_HABITACION)	580.45
Expresión lógica	---	servicio_adicional = S OR s ; Si (servicio_adicional)	Se cumple (Verdadero)
Costo final	---	costo_total + COSTO_EMPAQUE	780.45
resto	---	(numerohabitaciones) % 2	1

Resultado: La condición se cumple

El costo es 580.45

Servicio adicional: S

Costo por empaque: 200

Costo final: 780.45

Datos	**Entradas**	**Procesos**	**Salidas**
Distancia	25	---	---
Peso	896.88	---	
Volumen	8,88	---	---
Número habitaciones	5	---	---
Costo total	---	TARIFA_BASE + (distanciakm * COSTO_KM) + (peso_totalkg * COSTO_KG) + (volumen_m3 * COSTO_M3) + (numerohabitaciones * COSTO_HABITACION)	964.84
Expresión lógica	---	servicio_adicional = S OR s ; Si (servicio_adicional)	No se cumple (Falso)
Costo final	---	costo_total + COSTO_EMPAQUE	No aplica (964.84)

resto	---	(numerohabitaciones) % 2	1

Resultado: La condición se cumple
El costo es 964.84
Servicio adicional: N
Costo final: 964.84

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 254

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

```
asus@asus:~/Ej-254$ gcc Ej-254.c -o Ej-254 -lm && ./Ej-254
Ingrese la distancia (Km):11
Ingrese el peso total (Kg):530.89
Ingrese el volumen (m³):4.3
Ingrese el numero de habitaciones:3
El costo total es: 580.45
Desea servicio adicional, S/N
S
El costo por empaque es: 200.00
El costo final es: 780.45
La tarifa base es: 100.00
El costo por Km es: 1.00
El costo por Kg es: 0.50
El costo por m³ es: 30.00
El costo por habitación es: 25.00
El resto de una división adicional es: 1
asus@asus:~/Ej-254$ gcc Ej-254.c -o Ej-254 -lm && ./Ej-254
Ingrese la distancia (Km):25
Ingrese el peso total (Kg):896.88
Ingrese el volumen (m³):8.88
Ingrese el numero de habitaciones:5
El costo total es: 964.84
Desea servicio adicional, S/N
N
La tarifa base es: 100.00
El costo por Km es: 1.00
El costo por Kg es: 0.50
El costo por m³ es: 30.00
El costo por habitación es: 25.00
El resto de una división adicional es: 1
asus@asus:~/Ej-254$
```

Fig.20: Resultados de compilación del programa – 254.

- Enunciado del algoritmo

  255. Cree un algoritmo que simule el cálculo del salario neto de un trabajador. Lea el salario bruto (real), el número de cargas familiares (entero) y los años de antigüedad (entero). Use constantes simbólicas: BONO_CARGA = 50.0, BONO_ANTIGUEDAD = 30.0, DESCUENTO_ISSS = 0.03, DESCUENTO_AFP = 0.0725, LIMITE_ANTIGUEDAD

= 10. Calcule: bonos = cargas * BONO_CARGA + min(antiguedad, LIMITE_ANTIGUEDAD) * BONO_ANTIGUEDAD; descuentos = salario_bruto * (DESCUENTO_ISSS + DESCUENTO_AFP); salario_neto = salario_bruto + bonos - descuentos. Evalúe la expresión lógica (salario_neto > salario_bruto). Muestre por pantalla el desglose completo.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Salario bruto
  Cargas familiares
  Años antigüedad

- Procesos:

  bonos = (cargas * BONO_CARGA) + (antiguedad_valida * BONO_ANTIGUEDAD)
  descuentos = salario_bruto * (DESCUENTO_ISSS + DESCUENTO_AFP) ;
  salario_neto = salario_bruto + bonos – descuentos
  Si (salario_neto > salario_bruto)

- Datos de salida:

  Salario Neto
  Descuentos
  Desglose completo de datos

- Variables:

  salario_bruto = 0.0 →real
  cargas = 0 →entero
  antiguedad = 0 →entero
  antiguedad_valida = 0 →entero
  bonos = 0.0 →real
  descuentos = 0.0 →real
  salario_neto = 0.0 →real

- Constantes:

  BONO_CARGA = 50.0 →real
  BONO_ANTIGUEDAD = 30.0 →real
  DESCUENTO_ISSS = 0.03 →real
  DESCUENTO_AFP = 0.0725 →real
  LIMITE_ANTIGUEDAD = 10 →entero

- Diagrama de flujo

  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

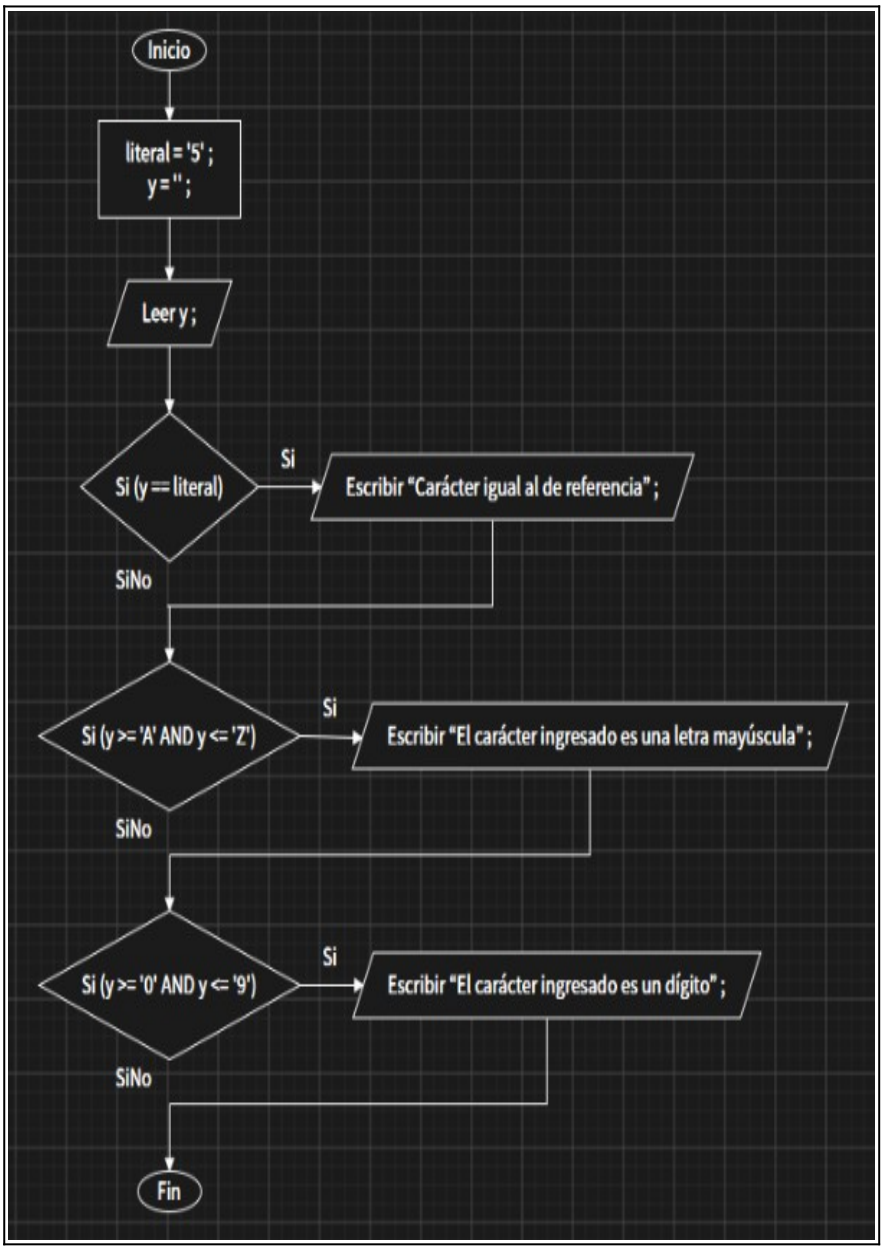


Fig.21: Diagrama de flujo en Draw.io – Algoritmo 255 .

- Pseudocódigo

```
Inicio
BONO_CARGA = 50.0 ;
BONO_ANTIGUEDAD = 30.0 ;
DESCUENTO_ISSS = 0.03 ;
DESCUENTO_AFP = 0.0725 ;
LIMITE_ANTIGUEDAD = 10 ;
```

```
salario_bruto = 0.0 ;
cargas = 0 ;
antiguedad = 0 ;
antiguedad_valida = 0 ;
bonos = 0.0 ;
descuentos = 0.0 ;
salario_neto = 0.0 ;
Leer salario_bruto ;
Leer cargas ;
Leer antiguedad ;
Si antiguedad < LIMITE_ANTIGUEDAD
antiguedad_valida = antiguedad ;
Sino
antiguedad_valida = LIMITE_ANTIGUEDAD ;
FinSi
bonos = (cargas * BONO_CARGA) + (antiguedad_valida * BONO_ANTIGUEDAD) ;
descuentos = salario_bruto * (DESCUENTO_ISSS + DESCUENTO_AFP) ;
salario_neto = salario_bruto + bonos - descuentos ;
Escribir "Salario bruto: " + salario_bruto + "$" ;
Escribir "Bonos: " + bonos + "$" ;
Escribir "Descuentos: " + descuentos + "$" ;
Escribir "Salario neto: " + salario_neto + "$" ;
Si (salario_neto > salario_bruto)
Escribir "Los bonos superan a los descuentos: el salario neto es mayor al salario bruto." ;
Sino
Escribir "Los descuentos superan a los bonos: el salario neto es menor o igual al salario
bruto." ;
FinSi
Fin
```

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Salario bruto	482	---	---
Cargas Familiares	4	---	
Años antiguedad	9	---	---
Bonos	---	(cargas * BONO_CARGA) + (antiguedad_valida * BONO_ANTIGUEDAD)	470
Descuentos	---	salario_bruto * (DESCUENTO_ISSS + DESCUENTO_AFP)	49.40

Salario neto	---	salario_bruto + bonos - descuentos	902.59

Resultado: Años Antiguedad: 9 (9 < LIMITE_ANTIGUEDAD)

Salario Neto: 902.59 $

Los bonos superan a los descuentos, el salario neto es mayor al salario bruto

Datos	Entradas	Procesos	Salidas
Salario bruto	1002	---	---
Cargas Familiares	5	---	
Años antiguedad	15	---	---
Bonos	---	(cargas * BONO_CARGA) + (antiguedad_valida * BONO_ANTIGUEDAD)	550
Descuentos	---	salario_bruto * (DESCUENTO_ISSS + DESCUENTO_AFP)	102.71
Salario neto	---	salario_bruto + bonos - descuentos	1449.30

Resultado: Años Antiguedad: 15 (15 > LIMITE_ANTIGUEDAD), por lo tanto se asigna 10

Salario Neto: 1449.30 $

Los bonos superan a los descuentos, el salario neto es mayor al salario bruto

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 255

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

```
asus@asus:~/Ej-255$ gcc Ej-255.c -o Ej-255 -lm && ./Ej-255
Ingrese el salario bruto: 482
Ingrese el número de cargas familiares: 4
Ingrese los años de antiguedad: 9
Salario bruto: 482.00 $
Bonos: 470.00 $
Descuentos: 49.40 $
Salario neto: 902.59 $
Los bonos superan a los descuentos, el salario neto es mayor al salario bruto
asus@asus:~/Ej-255$ gcc Ej-255.c -o Ej-255 -lm && ./Ej-255
Ingrese el salario bruto: 1002
Ingrese el número de cargas familiares: 5
Ingrese los años de antiguedad: 15
Salario bruto: 1002.00 $
Bonos: 550.00 $
Descuentos: 102.71 $
Salario neto: 1449.30 $
Los bonos superan a los descuentos, el salario neto es mayor al salario bruto
asus@asus:~/Ej-255$
```

Fig.22: Resultados de compilación del programa – 255.

- Enunciado del algoritmo

  301. Construya un algoritmo que lea un carácter desde teclado y lo almacene en la variable y. Use una constante literal '5' como carácter de referencia. Evalúe las expresiones lógicas: (y == '5'), (y >= 'A' AND y <= 'Z') y (y >= '0' AND y <= '9'). Muestre por pantalla un mensaje indicando si el carácter ingresado es igual al de referencia, si es letra mayúscula o si es dígito. Identifique el tipo de dato de cada variable.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  y (carácter)

- Procesos:

  Si (y == literal)
  Si (y >= 'A' AND y <= 'Z')
  SI (y >= '0' AND y <= '9')

- Datos de salida:

  Identificación del dato ingresado (carácter o dígito)

- Variables:
  y = ‘’ → carácter

- Constantes:
  literal = ‘5’ → caracter

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

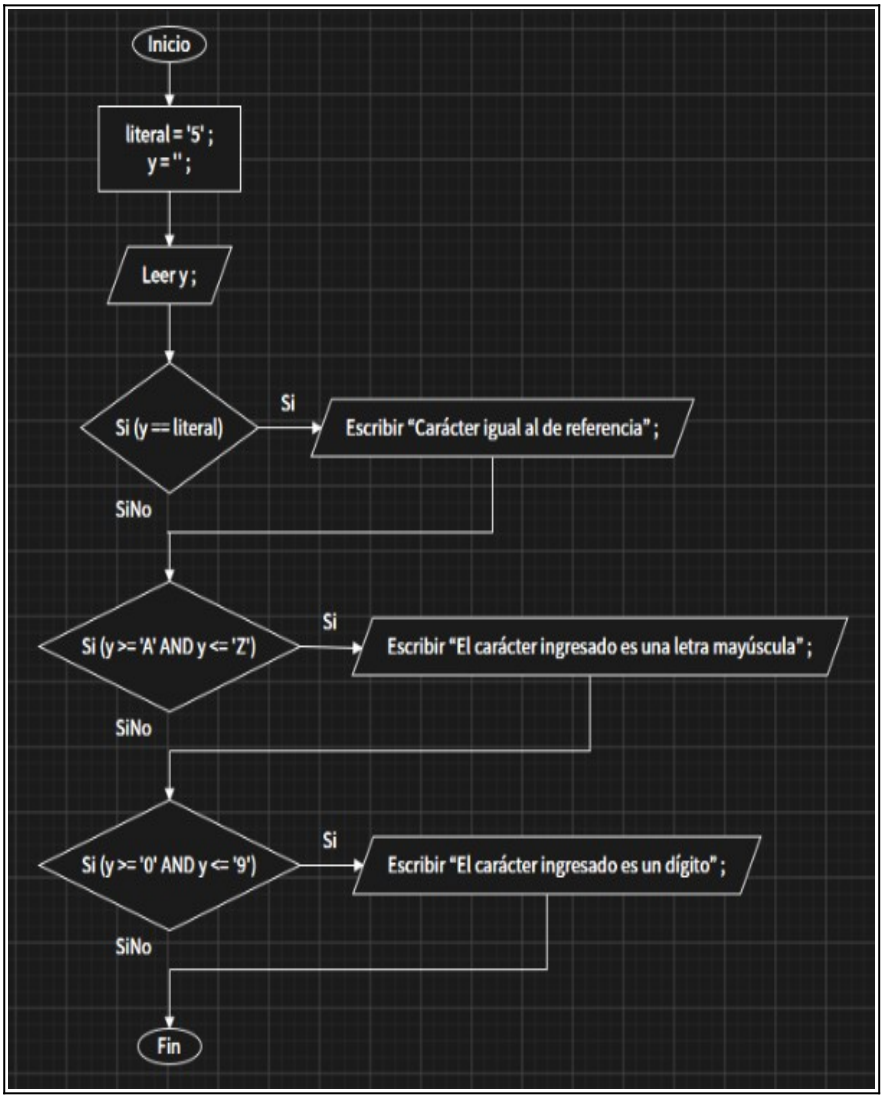


Fig.23: Diagrama de flujo en Draw.io – Algoritmo 301 .

- Pseudocódigo
  Inicio
  literal = ‘5’ ;
  y = ‘’ ;

```
Leer y ;
Si (y == literal)
Escribir “Carácter igual al de referencia” ;
FinSi
Si (y >= 'A' AND y <= 'Z')
Escribir “El carácter ingresado es una letra mayúscula” ;
FinSi
SI (y >= '0' AND y <= '9') ;
Escribir “El carácter ingresado es un dígito” ;
Finsi
Fin
```

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Y (carácter o dígito)	5	---	---
Expresión lógica	---	Si (y == literal)	Se cumple (Verdadero), Carácter igual al de referencia
Expresión lógica	---	Si (y >= 'A' AND y <= 'Z')	No se cumple (Falso)
Expresión lógica	---	Si (y >= '0' AND y <= '9')	Se cumple (Verdadero), es un dígito

Resultado: Primera condición verdadera, segunda condición falsa, tercera condición verdadera

Carácter igual al de referencia

El carácter ingresado es un dígito

Datos	Entradas	Procesos	Salidas
Y (carácter o dígito)	X	---	---
Expresión lógica	---	Si (y == literal)	No se cumple (Falso)
Expresión lógica	---	Si (y >= 'A' AND y <= 'Z')	Se cumple (Verdadero), es una letra mayúscula
Expresión lógica	---	Si (y >= '0' AND y <= '9')	No se cumple (Falso)

Resultado: Primera condición falsa, segunda condición verdadera, tercera condición falsa

El carácter ingresado es una letra mayúscula

Datos	Entradas	Procesos	Salidas
Y (carácter o dígito)	8	---	---
Expresión lógica	---	Si (y == literal)	No se cumple (Falso)
Expresión lógica	---	Si (y >= 'A' AND y <= 'Z')	No se cumple (Falso)

Expresión lógica	---	Si (y >= '0' AND y <= '9')	Se cumple (Verdadero), es un dígito

Resultado: Primera condición falsa, segunda condición falsa, tercera condición verdadera
El carácter ingresado es un dígito

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 301

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

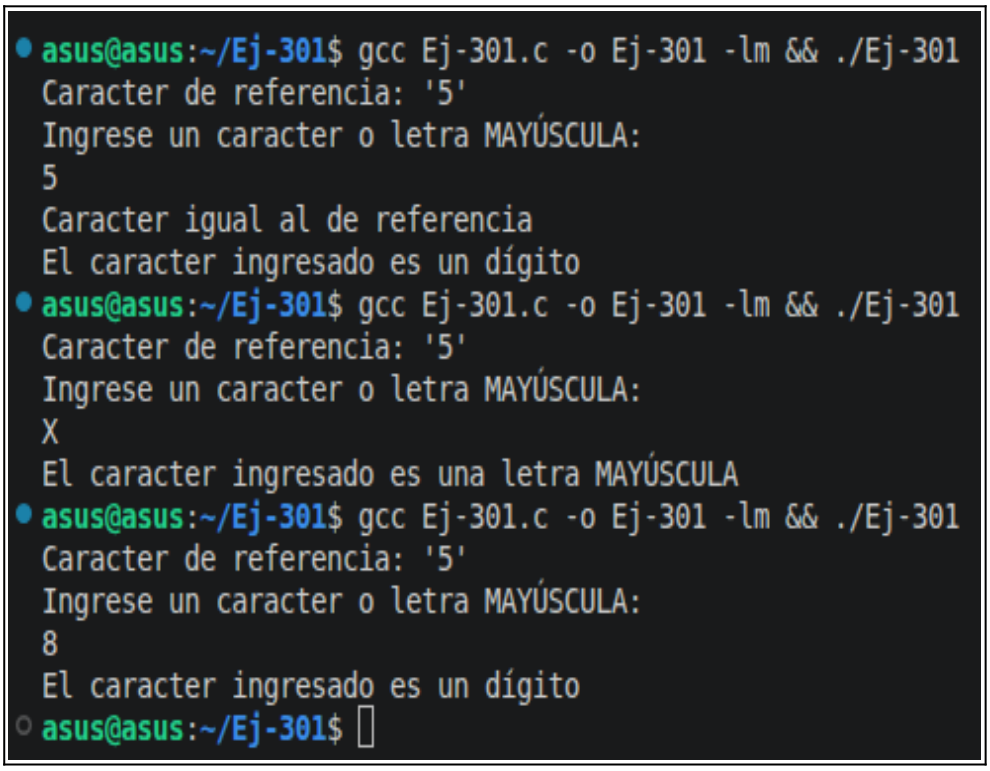


Fig.24: Resultados de compilación del programa – 201.

- Enunciado del algoritmo

  304. Construya un algoritmo que lea tres números enteros val1, num1 y m por teclado. Use una constante literal 3 para calcular el promedio. Asigne el resultado de la expresión aritmética ((val1 + num1 + m) / 3) a la variable promedio. Evalúe las expresiones relacionales (val1 > promedio), (num1 > promedio) y (m > promedio). Muestre por pantalla cuáles de los tres valores están por encima del promedio.

- Análisis del problema
  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:
  Valor 1 (val1)
  Valor 2 (num1)
  Valor 3 (m)

- Procesos:
  ((val1 + num1 + m) / 3)
  (val1 > promedio)
  (num1 > promedio)
  (m > promedio)

- Datos de salida:
  El valor que esta por encima del promedio

- Variables:
  val1 = 0 →entero
  num1 = 0 →entero
  m = 0 →entero
  promedio = 0 →entero

- Constantes:
  constante = 3 →entero

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

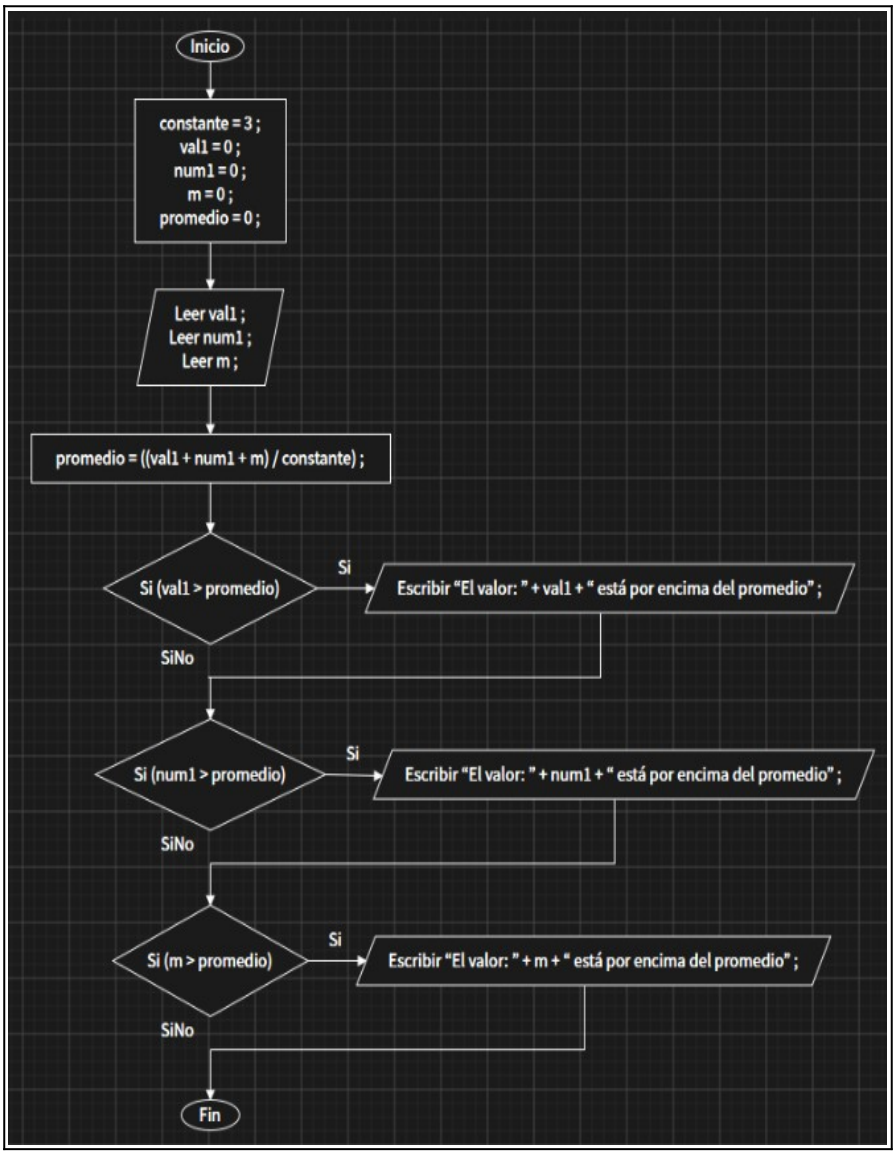


Fig.25: Diagrama de flujo en Draw.io – Algoritmo 304 .

- Pseudocódigo

  Inicio
  constante = 3 ;
  val1 = 0 ;
  num1 = 0 ;
  m = 0 ;
  promedio = 0 ;
  Leer val1;
  Leer num1;
  Leer m ;
  promedio = ((val1 + num1 + m)/ constante) ;
  Si (val1 > promedio) Escribir “El valor: ” + val1 + “está por encima del promedio” ;
  FinSi

Si (num1 > promedio)
Escribir “El valor: ” + num1 + “está por encima del promedio” ;
FinSi
Si (m > promedio)
Escribir “El valor: ” + m + “está por encima del promedio” ;
FinSi
Fin

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Entero 1 (val1)	78	---	---
Entero 1 (num1)	9	---	---
Entero 1 (m)	16	---	---
promedio	---	promedio = ((val1 + num1 + m)/ constante)	---
Expresión lógica	---	(val1 > promedio)	Se cumple (Verdadero), el valor 78 está por encima del promedio
Expresión lógica	---	(num1 > promedio)	No se cumple (Falso)
Expresión lógica	---	(m > promedio)	No se cumple (Falso)

Resultado: Primera condición verdadera, segunda y tercera condición falsa
El valor: 78, está por encima del promedio

Datos	Entradas	Procesos	Salidas
Entero 1 (val1)	18	---	---
Entero 1 (num1)	5	---	---
Entero 1 (m)	11	---	---
promedio	---	promedio = ((val1 + num1 + m)/ constante)	---
Expresión lógica	---	(val1 > promedio)	Se cumple (Verdadero), el valor 18 está por encima del promedio
Expresión lógica	---	(num1 > promedio)	No se cumple (Falso)
Expresión lógica	---	(m > promedio)	No se cumple (Falso)

Resultado: Primera condición verdadera, segunda y tercera condición falsa
El valor: 18, está por encima del promedio

- Codificación en lenguaje C.
  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.
  Ejercicio – 304

- Salida del programa

Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

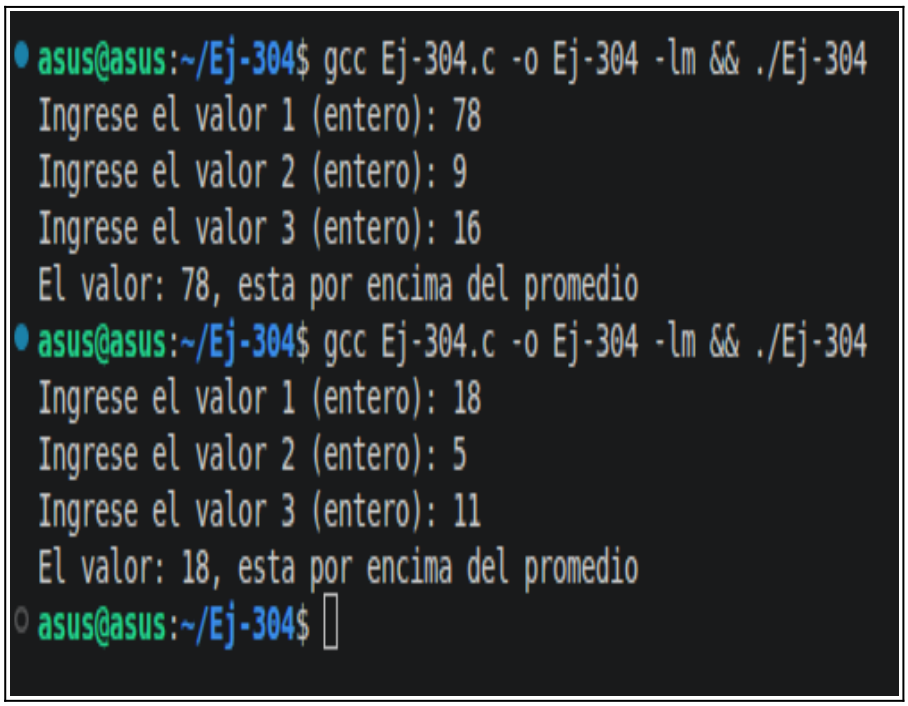


Fig.26: Resultados de compilación del programa – 304.

- Enunciado del algoritmo

  327. Construya un algoritmo que lea dos números reales num2 y y desde el teclado. Use una constante simbólica llamada UMBRAL con valor 0.5 para representar un factor de conversión. Calcule la expresión aritmética: ((num2 + y) * UMBRAL) / 2. Asigne el resultado a una variable llamada resultado. Muestre por pantalla el valor obtenido junto con un mensaje descriptivo. Explique qué tipo de dato simple debe usar para cada variable y por qué.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Valor 1 (num2)
  Valor 2 (y)

- Procesos:

  ((num2 + y) * UMBRAL) / 2

- Datos de salida:

Resultado

- Variables:
  num2 = 0.0 → real
  y = 0.0 → real
  resultado = → real

- Constantes:
  UMBRAL = 0.5 → real

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, paralelogramos y flechas.

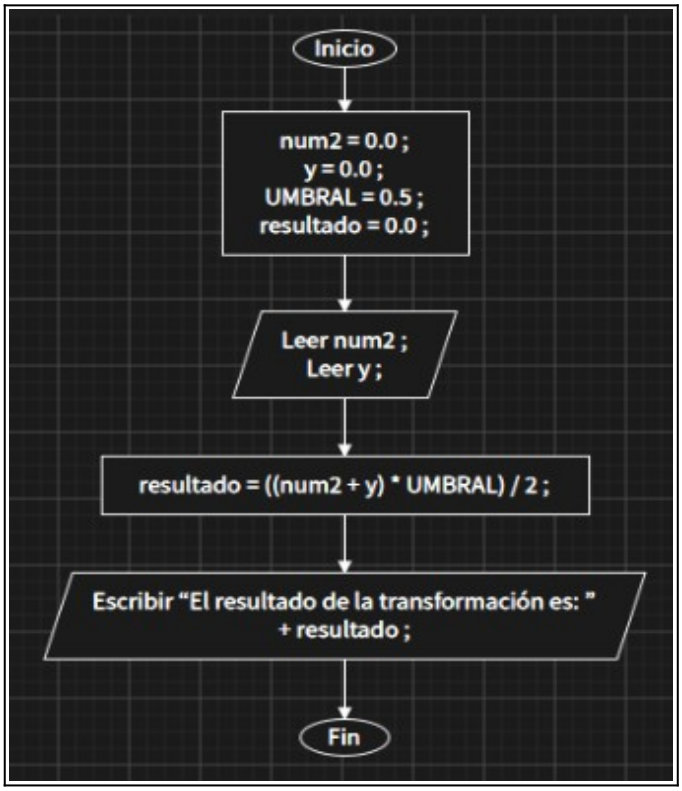


Fig.27: Diagrama de flujo en Draw.io – Algoritmo 327 .

- Pseudocódigo
  Inicio
  num2 = 0.0 ;
  y = 0.0 ;
  UMBRAL = 0.5 ;
  resultado = 0.0 ;
  Leer num2 ;
  Leer y ;
  resultado = ((num2 + y) * UMBRAL) / 2 ;

Escribir “El resultado de la transformación es: ” + resultado ;
Fin

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Real 1 (num2)	57.89	---	---
Real 2 (y)	23.014	---	---
resultado	---	((num2 + y) * UMBRAL) / 2	20.23

Resultado: Factor de transformación UMBRAL = 0.5
El resultado de la transformación es 20.23

Datos	Entradas	Procesos	Salidas
Real 1 (num2)	45	---	---
Real 2 (y)	83.24	---	---
resultado	---	((num2 + y) * UMBRAL) / 2	32.06

Resultado: Factor de transformación UMBRAL = 0.5
El resultado de la transformación es 32.06

- Codificación en lenguaje C.
  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.
  Ejercicio – 327

- Salida del programa
  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

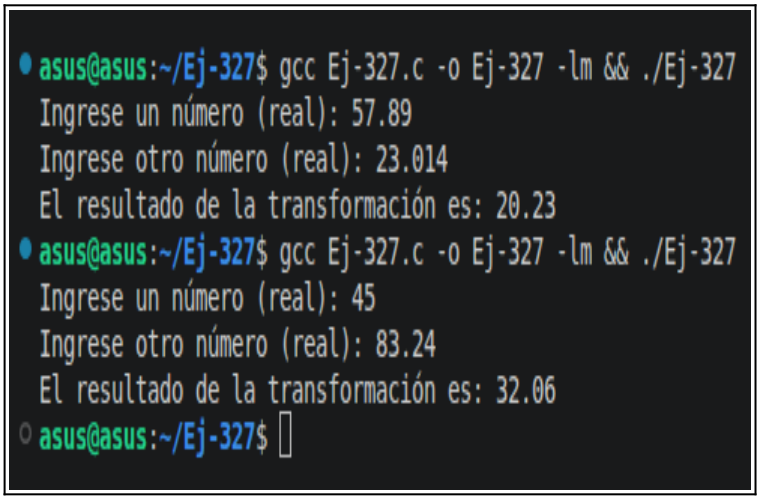


Fig.28: Resultados de compilación del programa – 327.

- Enunciado del algoritmo

  401. Diseñe un algoritmo que declare una variable a (entero) y lea su valor por teclado. Use una constante simbólica LIMITE = 50. Evalúe la expresión lógica (a > 0 AND a < LIMITE). Asigne el resultado a una variable booleana es_valido. Evalúe también la expresión relacional (a == LIMITE / 2). Muestre por pantalla ambos resultados identificando cuál es dato y cuál es información.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Valor 1 (a)

- Procesos:

  Expresión lógica (a > 0 AND a < LIMITE)
  Expresión relacional (a == LIMITE / 2)

- Datos de salida:

  Número válido o invalido
  Número ingresado igual o no igual a la mitad del límite

- Variables:

  a = 0 → entero
  es_valido = false → booleano

- Constantes:

  Limite = 50 → entero

- Diagrama de flujo

  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

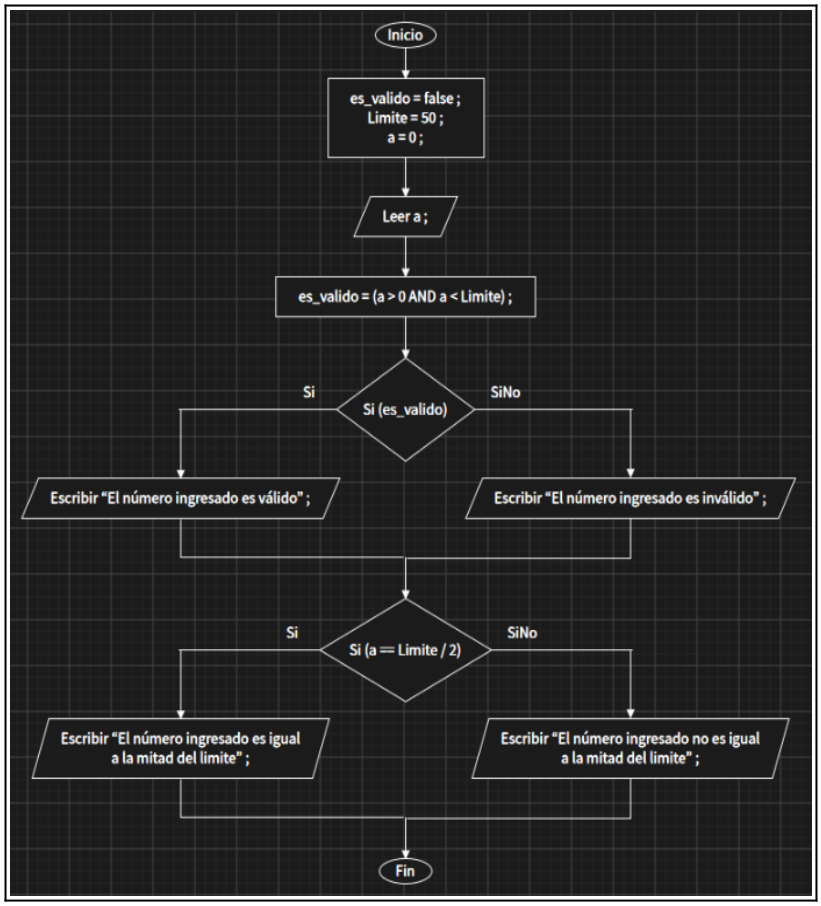


Fig.29: Diagrama de flujo en Draw.io – Algoritmo 401.

- Pseudocódigo

```
Inicio
es_valido = false ;
Limite = 50 ;
a = 0 ;
Leer a ;
es_valido = (a > 0 AND a < Limite) ;
Si (es_valido)
Escribir "El número ingresado es válido" ;
Sino
Escribir "El número ingresado es inválido" ;
FinSi
Si (a == Limite/2)
Escribir "El número ingresado es igual a la mitad del limite " ;
Sino
Escribir "El número ingresado no es igual a la mitad del limite " ;
FinSi
Fin
```

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Entero 1 (a)	25	---	---
Expresión lógica	---	es_valido = (a > 0 AND a < Limite)	Se cumple (Verdadero)
Expresión lógica		Si (es_valido)	Número ingresado válido
Expresión lógica	---	Si (a == Limite/2)	Se cumple (Verdadero), número ingresado es igual a la mitad del limite

Resultado: Todas las condiciones se cumplen
El número ingresado es válido
El número ingresado es igual a la mitad del límite.

Datos	Entradas	Procesos	Salidas
Entero 1 (a)	148	---	---
Expresión lógica	---	es_valido = (a > 0 AND a < Limite)	No se cumple (Falso)
Expresión lógica		Si (es_valido)	Número ingresado inválido
Expresión lógica	---	Si (a == Limite/2)	No se cumple (Falso), número ingresado no es igual a la mitad del limite

Resultado: Ninguna condición se cumple
El número ingresado es inválido
El número ingresado no es igual a la mitad del límite.

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.
  Ejercicio – 401

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

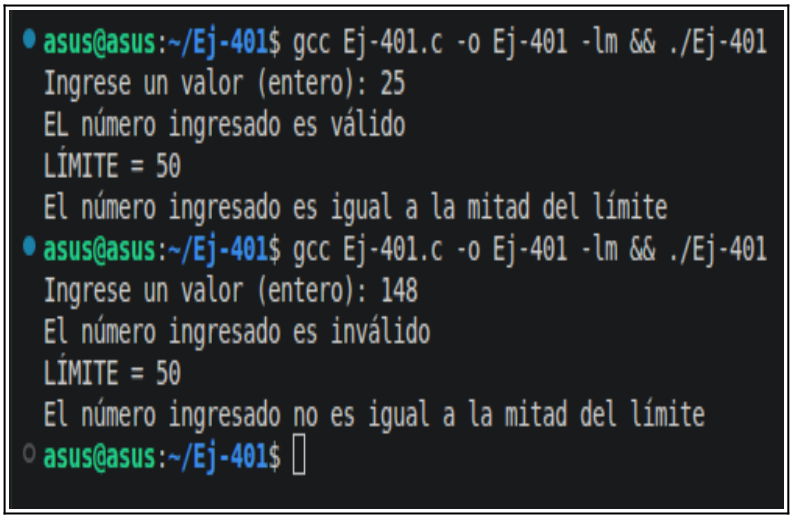


Fig.30: Resultados de compilación del programa – 401.

- Enunciado del algoritmo

  406. Construya un algoritmo que declare una variable entera z. Asigne el valor literal 50. Lea otra variable entera dato2 por teclado. Use una constante simbólica MAXIMO = 100. Evalúe la expresión lógica (z + dato2 <= MAXIMO AND z > 0 AND dato2 > 0). Calcule la expresión aritmética (z * dato2) y asígnela a producto. Muestre por pantalla el producto y el resultado de la expresión lógica.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Valor 1 (dato2)

- Procesos:

  Expresión lógica (z + dato2 <= MAXIMO AND z > 0 AND dato2 > 0)
  Expresión aritmética (z * dato2)

- Datos de salida:

  Valor ingresado válido o invalido
  producto

- Variables:

  producto = 0 → entero
  dato2 = 0 → entero

- Constantes:
  maximo = 100 → entero
  z = 50 → entero

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

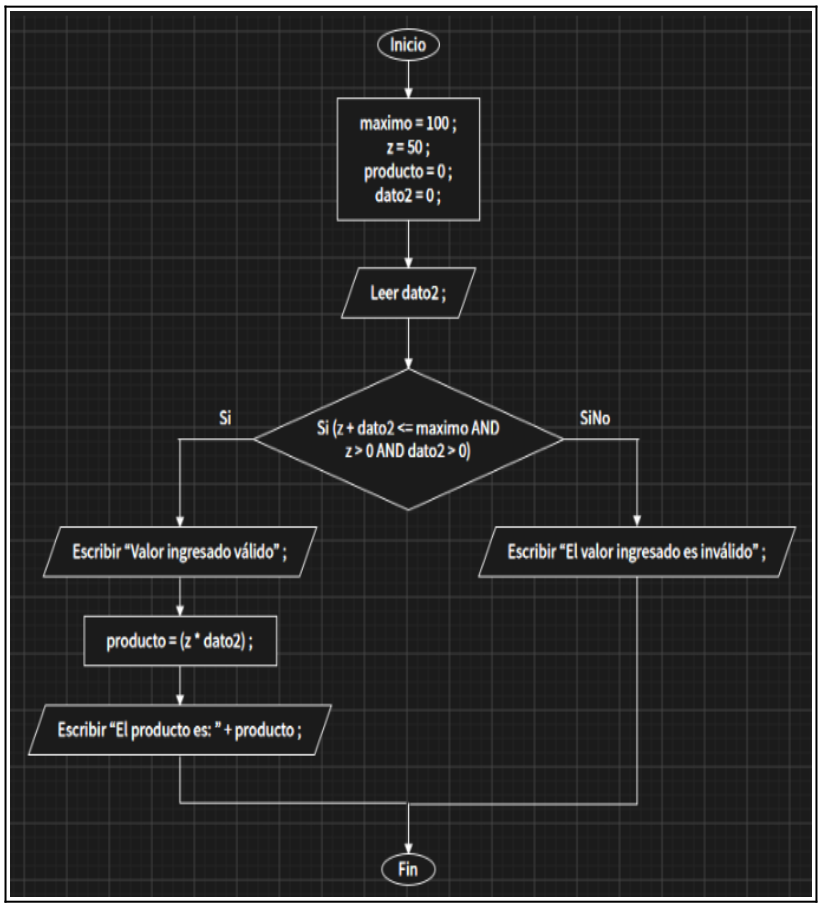


Fig.31: Diagrama de flujo en Draw.io – Algoritmo 406.

- Pseudocódigo
  Inicio
  maximo = 100 ;
  z = 50 ;
  producto = 0 ;
  dato2 = 0 ;
  Leer dato2 ;
  Si (z + dato2 <= maximo AND z > 0 AND dato2 > 0)
  Escribir “Valor ingresado válido” ;
  producto = (z * dato2) ;
  Escribir “El producto es: ” + producto ;
  Sino

Escribir “El valor ingresado es inválido” ;
FinSi
Fin

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Entero 1 (dato2)	83	---	---
Expresión lógica	---	Si (z + dato2 <= maximo AND z > 0 AND dato2 > 0)	No se cumple (Falso)
producto	---	(z * dato2)	No aplica

Resultado: La primera expresión de la condición no se cumple, ya que z + dato2 = 133, 133 > 100, y aunque el resto de expresiones se cumplen la condición es falsa
El valor ingresado es inválido

Datos	Entradas	Procesos	Salidas
Entero 1 (dato2)	50	---	---
Expresión lógica	---	Si (z + dato2 <= maximo AND z > 0 AND dato2 > 0)	Se cumple (Verdadero)
producto	---	(z * dato2)	2500

Resultado: Todas las expresiones de la condición se cumplen, es verdadera
El valor ingresado es válido
El producto es 2500

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.
  Ejercicio – 406

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

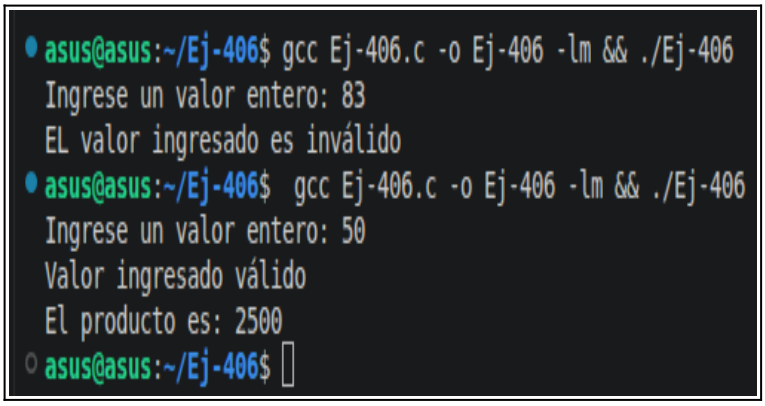


Fig.32: Resultados de compilación del programa – 406.

- Enunciado del algoritmo

  431. Declare tres variables enteras llamadas x, a y val2. Solicite al usuario que ingrese tres números enteros por teclado (lectura) y almacénelos en dichas variables. Calcule la suma, el producto y el promedio (expresión aritmética) usando una constante literal 25 para representar la cantidad de elementos. Finalmente, muestre por pantalla (escritura) un mensaje informativo que indique: "La suma es X, el producto es Y y el promedio es Z". Identifique claramente el tipo de dato de cada variable (entero, real) y justifique por qué el promedio debe declararse como real.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Valor 1 (x)
  Valor 2 (a)
  Valor 3 (val2)

- Procesos:

  X = x + a + val2
  Y = (x * a * val2)
  Z = (x + a + val2) / cantidad_elementos

- Datos de salida:

  Suma
  Producto
  Promedio

- Variables:
  X = 0 →entero
  Y = 0 →entero
  Z = 0.0 →real (porque al dividir los números ingresados no siempre saldrá entero, también puede salir decimal)
  x = 0 →entero
  a = 0 →entero
  val2 = 0 →entero

- Constantes:
  cantidad_elementos = 3 →entero

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, paralelogramos y flechas.

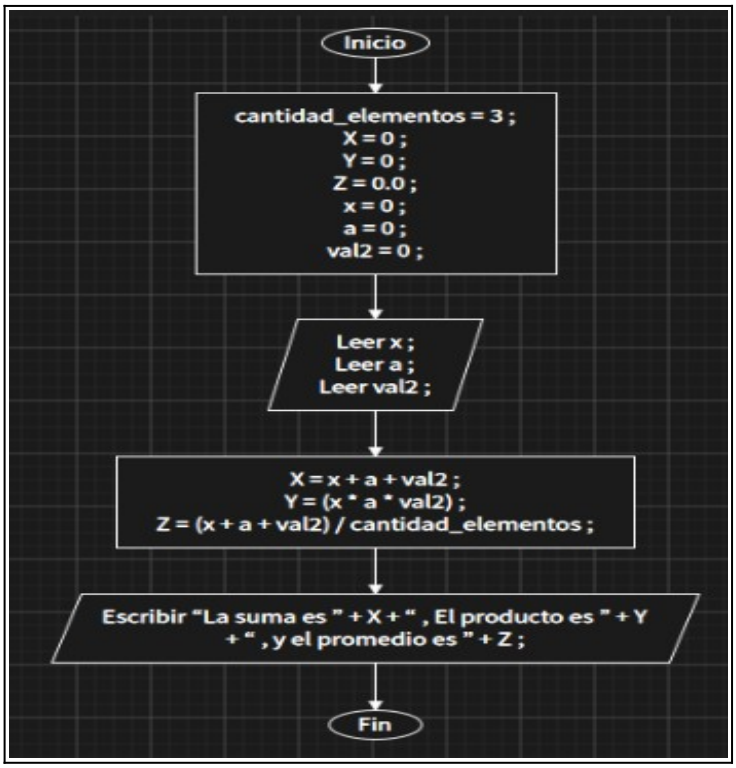


Fig.33: Diagrama de flujo en Draw.io – Algoritmo 431.

- Pseudocódigo
  Inicio
  cantidad_elementos = 3 ;
  X = 0 ;
  Y = 0 ;
  Z = 0.0 ;
  x = 0 ;

a = 0 ;
val2 = 0 ;
Leer x ;
Leer a ;
Leer val2 ;
X = x + a + val2 ;
Y = (x * a * val2) ;
Z = (x + a + val2) / cantidad_elementos ;
Escribir “La suma es ” + X + “ , El producto es ” + Y + “ , y el promedio es ” + Z ;
Fin

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Entero 1 (x)	23	---	---
Entero 2 (a)	11	---	---
Entero 3 (val2)	53	---	---
Suma (X)	---	x + a + val2	87
Producto (Y)	---	(x * a * val2)	13 409
Promedio (Z)	---	(x + a + val2) / cantidad_elementos	29

Resultado:
La suma es 87: ,el producto es: 13 409, y el promedio es: 29

Datos	Entradas	Procesos	Salidas
Entero 1 (x)	1	---	---
Entero 2 (a)	2	---	---
Entero 3 (val2)	4	---	---
Suma (X)	---	x + a + val2	7
Producto (Y)	---	(x * a * val2)	8
Promedio (Z)	---	(x + a + val2) / cantidad_elementos	2.33

Resultado:
La suma es : ,el producto es: 8, y el promedio es: 2.33

- Codificación en lenguaje C.
  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.
  Ejercicio – 431

- Salida del programa

Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

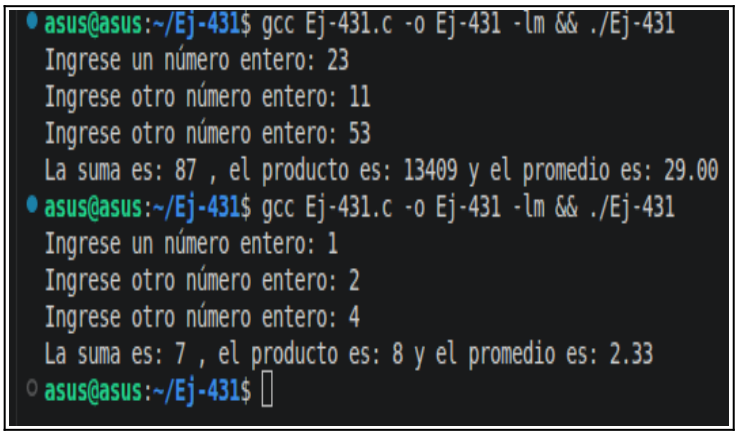


Fig.34: Resultados de compilación del programa – 431.

Nota: En este ejercicio se cambio la constante literal 25 por 3, para que el ejercicio tenga lógica de redacción y por tanto de solución.

- Enunciado del algoritmo

  438. Construya un algoritmo que lea el sueldo base (real) de un trabajador. Use una constante simbólica BONO = 1.5 (porcentaje de bono) y una constante literal 100 para el cálculo. Calcule el bono con la expresión aritmética (sueldo * BONO / 100) y el sueldo total con (sueldo + bono). Evalúe la expresión relacional (sueldo_total > sueldo). Muestre por pantalla el desglose completo.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Sueldo base

- Procesos:

  Expresión aritmética (sueldo * BONO / 100)
  (sueldo + bono)
  Expresión relacional (sueldo_total > sueldo)

- Datos de salida:

  Bono total
  Sueldo total
  Desglose completo

- Variables:
  sueldo_base = 0.0 →real
  bono_total = 0.0 →real
  sueldo_total = 0.0 →real

- Constantes:
  BONO = 1.5 →real
  CONSTANTE = 100 →entero

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.

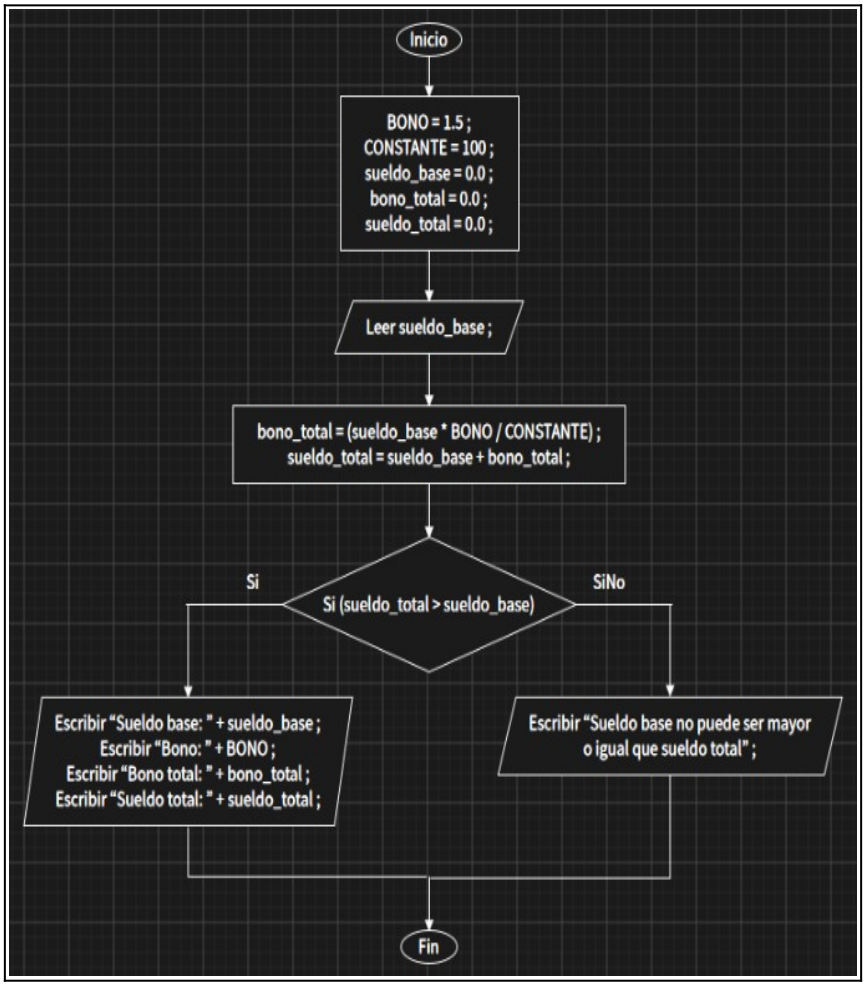


Fig.35: Diagrama de flujo en Draw.io – Algoritmo 438.

- Pseudocódigo
  BONO = 1.5 ;
  CONSTANTE = 100 ;
  sueldo_base = 0.0 ;

```
bono_total = 0.0 ;
sueldo_total = 0.0 ;
Leer sueldo_base ;
bono_total = (sueldo_base * BONO / CONSTANTE) ;
sueldo_total = sueldo_base + bono_total  ;
Si (sueldo_total > sueldo_base)
Escribir “Sueldo base : ” + sueldo_base ;
Escribir “Bono: ” + BONO ;
Escribir “Bono total: ” + bono_total ;
Escribir “Sueldo total: “ + sueldo_total ;
Sino
Escribir “Sueldo base no puede ser mayor o igual que sueldo total” ;
FinSi
Fin
```

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Sueldo base	482.50	---	---
Bono total	---	(sueldo_base * BONO / CONSTANTE)	7.24
Sueldo total	---	sueldo_base + bono_total	489.74
Expresión lógica	---	(sueldo_total > sueldo_base)	Se cumple (Verdadero)

Resultado: La condición se cumple

Bono total: 7.24

Sueldo total: 489.74

Datos	Entradas	Procesos	Salidas
Sueldo base	300	---	---
Bono total	---	(sueldo_base * BONO / CONSTANTE)	4.50
Sueldo total	---	sueldo_base + bono_total	304.50
Expresión lógica	---	(sueldo_total > sueldo_base)	Se cumple (Verdadero)

Resultado: La condición se cumple

Bono total: 4.50

Sueldo total: 304.50

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.

  Ejercicio – 438

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

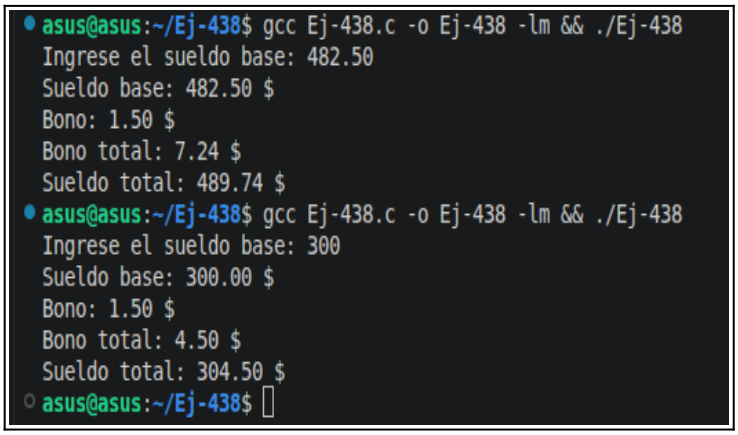


Fig.36: Resultados de compilación del programa – 438.

- Enunciado del algoritmo

  529. Construya un algoritmo que lea la velocidad en m/s (real) y la convierta a km/h. Use una constante simbólica CONVERSION = 3.6. Calcule con la expresión aritmética (velocidad_ms * CONVERSION) y asígnela a velocidad_kmh. Evalúe la expresión relacional (velocidad_kmh > 100) para identificar si supera el límite de velocidad. Muestre por pantalla el resultado con un mensaje claro.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Velocidad m/s

- Procesos:

  Expresión aritmética (velocidad_ms * CONVERSION)
  Expresión relacional (velocidad_kmh > 100)

- Datos de salida:

  Velocidad Km/h
  Limite de velocidad superado o no superado

- Variables:
  velocidad_ms = 0.0 →real
  velocidad_kmh = 0.0 →real

- Constantes:
  CONVERSION = 3.6 →real

- Diagrama de flujo
  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, rombos, paralelogramos y flechas.



Fig.37: Diagrama de flujo en Draw.io – Algoritmo 529.

- Pseudocódigo

```
Inicio
CONVERSION = 3.6 ;
velocidad_ms = 0.0 ;
velocidad_kmh = 0.0 ;
Leer velocidad_ms ;
velocidad_kmh = (velocidad_ms * CONVERSION) ;
Escribir “La velocidad en km/h es: ” + velocidad_kmh ;
Si (velocidad_kmh > 100)
Escribir “Límite de velocidad superado” ;
Sino
```

Escribir “Límite de velocidad no superado” ;
FinSi
Fin

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Velocidad m/s	900.26	---	---
Velocidad km/h	---	(velocidad_ms * CONVERSION)	3240.94
Expresión lógica	---	Si (velocidad_kmh > 100)	Se cumple (Verdadero), límite superado

Resultado: La condición se cumple, superó el límite de velocidad
La velocidad en km/h es: 3240.94
Límite de velocidad superado

Datos	Entradas	Procesos	Salidas
Velocidad m/s	20.57	---	---
Velocidad km/h	---	(velocidad_ms * CONVERSION)	74.05
Expresión lógica	---	Si (velocidad_kmh > 100)	No se cumple (Falso), límite no superado

Resultado: La condición no se cumple, no superó el límite de velocidad
La velocidad en km/h es: 74.05
Límite de velocidad no superado

- Codificación en lenguaje C.

  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.
  Ejercicio – 529

- Salida del programa

  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

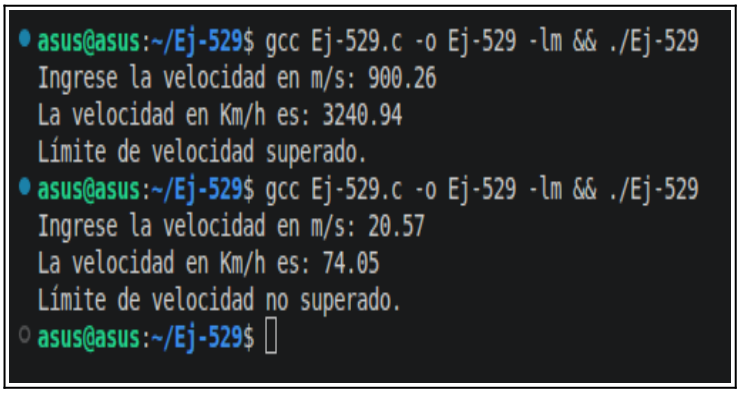


Fig.38: Resultados de compilación del programa – 529.

- Enunciado del algoritmo

  532. Construya un algoritmo que lea dos números reales dato1 y n desde el teclado. Use una constante simbólica llamada CONSTANTE con valor 0.1 para representar un factor de conversión. Calcule la expresión aritmética: ((dato1 + n) * CONSTANTE) / 2. Asigne el resultado a una variable llamada resultado. Muestre por pantalla el valor obtenido junto con un mensaje descriptivo. Explique qué tipo de dato simple debe usar para cada variable y por qué.

- Análisis del problema

  Identificar y describir: datos de entrada, proceso (fórmulas u operaciones), datos de salida y variables a utilizar con su tipo de dato.

- Datos de entrada:

  Valor 1 (dato1)
  Valor 2 (n)

- Procesos:

  Expresión aritmética: ((dato1 + n) * CONSTANTE) / 2

- Datos de salida:

  Resultado

- Variables:

  resultado = 0.0 → real
  dato1 = 0.0 → real
  n = 0.0 → real

- Constantes:

  CONSTANTE = 0.1 → real

- Diagrama de flujo

  El diagrama de flujo fue realizado en Draw.io usando las figuras correspondientes como óvalos, rectángulos, paralelogramos y flechas.

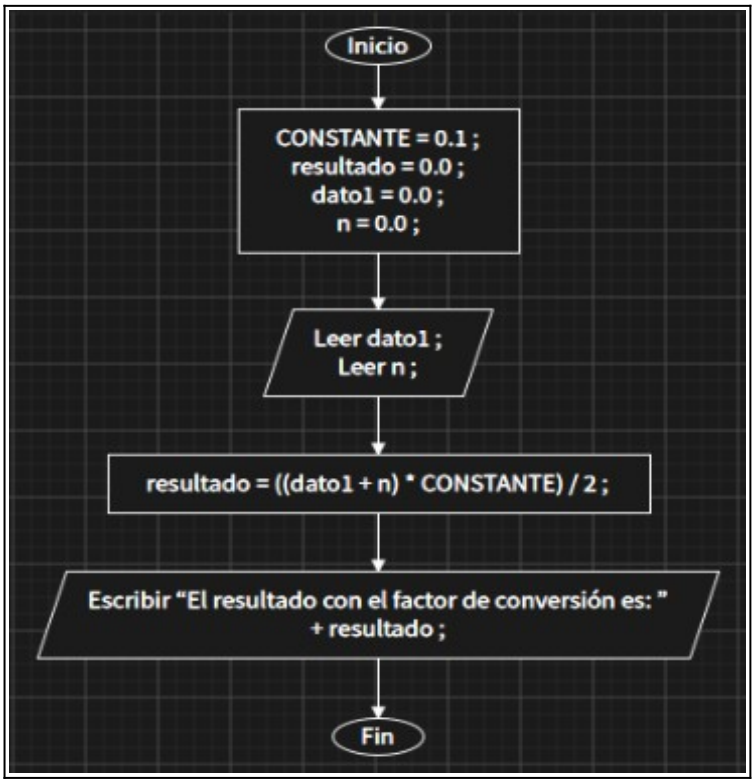


Fig.39: Diagrama de flujo en Draw.io – Algoritmo 532.

- Pseudocódigo

```
Inicio
CONSTANTE = 0.1 ;
resultado = 0.0 ;
dato1 = 0.0 ;
n = 0.0 ;
Leer dato1 ;
Leer n ;
resultado = ((dato1 + n) * CONSTANTE) / 2 ;
Escribir “El resultado con el factor de conversión es: ” + resultado ;
Fin
```

- Prueba de escritorio

Datos	Entradas	Procesos	Salidas
Real 1 (dato1)	9.98	---	---
Real 2 (n)	7.25	---	---
resultado	---	((dato1 + n) * CONSTANTE) / 2	0.86

Resultado:
El resultado con el factor de conversión (0.1) es: 0.86

Datos	Entradas	Procesos	Salidas
Real 1 (dato1)	5	---	---
Real 2 (n)	6.66	---	---
resultado	---	((dato1 + n) * CONSTANTE) / 2	0.58

Resultado:
El resultado con el factor de conversión (0.1) es: 0.58

- Codificación en lenguaje C.
  Se incluye aquí en enlace del repositorio digital de GitHub donde de encuentra el código escrito en lenguaje C.
  Ejercicio – 532

- Salida del programa
  Se incluye aquí una captura del resultado de la compilación del código (salida del programa), donde se ve que los resultados obtenidos (pruebas de escritorio) coinciden con los resultados esperados (compilación – ejecución del código).

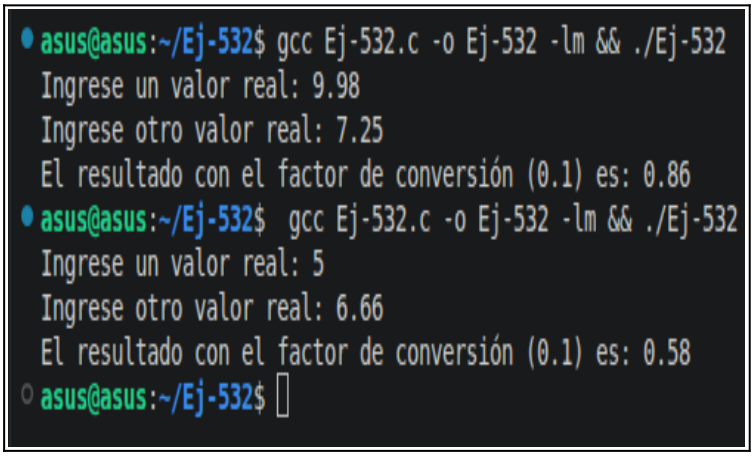


Fig.40: Resultados de compilación del programa – 532.

5. Conclusiones

- Se desarrolló todos los algoritmos propuestos por el docente de manera satisfactoria, aplicando todos los conceptos aprendidos en clase para la solución de un problema, los cuales comprenden análisis del problema, pseudocódigo, diagrama de flujo, pruebas de escritorio, escritura y compilación del código en lenguaje C.
- El orden de los pasos para el desarrollo de un algoritmo se respetaron siempre, sin ignorar, omitir o no verificar cada paso, lo cual ayudo a una rápida solución del problema planteado.

- Este tipo de trabajos aunque arduos, son la base fundamental para comprender los conceptos básicos de la programación, desarrollando habilidades básicas como la lógica de programación, lógica matemática, bases para aprender de manera más sencilla otros lenguajes de programación, y resolver problemas de manera mas eficiente.

6. Recomendaciones

- Recomiendo a la persona encargada de plantear los ejercicios para los estudiantes (docente), verificar si los ejercicios asignados tienen lógica, una correcta redacción y datos suficientes para su resolución, ya que algunos ejercicios era difíciles de comprenderlos y realizarlos por la ambigüedad que presentaba su enunciado, datos insuficientes u operaciones y procedimientos sin lógica, además muchos ejercicios se repetían para los estudiantes por lo cual la resolución de los mismos fue más tardada, más difícil de comprender y ejercicios que era mejor omitir antes que intentar resolverlos.
- Recomiendo a los estudiantes no omitir ningún paso para la resolución de un algoritmo, ya que todos cumplen un rol importante en el procedimiento, además, verificar la sintaxis del pseudocódigo y escritura del programa, una correcta revisión y detección de errores puede ahorrar mucho tiempo en la realización del trabajo.
- Recomiendo a los estudiantes no usar ningún tipo de ayuda de herramientas de Inteligencia Artificial, debido a que la dependencia excesiva de estas herramientas no permitirá desarrollar las habilidades necesarias de programación, si se tienen algún problema en una sección de código se puede apoyar en tutoriales de YouTube o guías del docente (en este caso Documento de códigos útiles en C, publicado por el docente en el EVA- Entrono Virtual de Aprendizaje).

7. Anexos

Se incluye el enlace general al repositorio digital de GitHub.

Enlace GitHub

Rúbrica de Evaluación.

Se aplica a cada algoritmo desarrollado.

Criterio	Excelente	Bueno	Regular	Insuficiente	Punt.
Análisis del problema (0,5 pts)	**(0,5)** Identifica correctamente entradas, proceso, salidas y variables con su tipo de dato.	**(0,4)** Identifica entradas, proceso y salidas con errores mínimos.	**(0,2)** Identifica parcialmente los elementos del análisis.	**(0)** No presenta el análisis o este es incorrecto.	
Diagrama de flujo (2,0 pts)	**(2,0)** Simbología correcta y flujo lógico completo; el algoritmo resuelve el problema.	**(1,5)** Simbología mayormente correcta; lógica con pequeños errores que no afectan el resultado.	**(1,0)** Errores de simbología y/o lógica que afectan parcialmente la solución.	**(0–0,5)** Diagrama incompleto, con lógica incorrecta o no presenta.	
Pseudocódigo (1,5 pts)	**(1,5)** Estructura completa y sintaxis adecuada; corresponde al diagrama de flujo.	**(1,2)** Estructura completa con errores menores de sintaxis.	**(0,8)** Estructura incompleta o con errores de sintaxis frecuentes.	**(0–0,4)** Pseudocódigo incorrecto o no presenta.	
Prueba de escritorio (2,0 pts)	**(2,0)** Tabla completa, ordenada y con resultados correctos; usa mínimo dos casos de prueba.	**(1,5)** Tabla completa con algún error de cálculo menor.	**(1,0)** Tabla incompleta o con varios errores que no validan el algoritmo.	**(0–0,5)** No presenta la prueba de escritorio o está totalmente errada.	
Codificación en C (2,0 pts)	**(2,0)** Código funcional, compilado, comentado, con buenas prácticas e indentación.	**(1,5)** Código funcional con errores menores de estilo o comentarios.	**(1,0)** Código con errores que impiden compilar o lógica incorrecta.	**(0–0,5)** Código no funcional o no presenta.	
Salida del programa (1,0 pto)	**(1,0)** Incluye captura de la ejecución; los resultados coinciden con la prueba de escritorio.	**(0,8)** Incluye captura con diferencias mínimas respecto a la prueba de escritorio.	**(0,5)** Incluye captura pero no coincide con la prueba de escritorio.	**(0–0,3)** No incluye evidencia de la ejecución.	
Conclusiones (0,5 pts)	**(0,5)** Conclusiones bien redactadas, claras y coherentes; de elaboración propia y reflejan aprendizaje y reflexión sobre el proceso.	**(0,4)** Conclusiones adecuadas con errores menores de redacción u ortografía.	**(0,2)** Conclusiones superficiales, muy breves o con deficiencias de redacción.	**(0)** No presenta conclusiones, son copiadas o generadas con IA.	
Presentación y redacción (0,5 pts)	**(0,5)** Documento ordenado, con portada, formato claro y redacción adecuada.	**(0,4)** Documento ordenado con errores menores de formato.	**(0,2)** Documento desordenado o con deficiencias de formato.	**(0)** Documento sin estructura ni presentación adecuada.	
PUNTAJE TOTAL POR ALGORITMO					**/ 10**

Escala de calificación

9,0 – 10	Domina el desarrollo de algoritmos
8,0 – 8,9	Alcanza los aprendizajes esperados
7,0 – 7,9	Está próximo a alcanzar los aprendizajes
Menos de 7,0	No alcanza los aprendizajes esperados

Observaciones adicionales

- Si se desarrollan N algoritmos, la nota final será el promedio de los puntajes obtenidos en cada uno.
- La coherencia entre el diagrama de flujo, el pseudocódigo, la prueba de escritorio, el código C y las conclusiones será un aspecto clave de la evaluación.
- El uso de inteligencia artificial o la copia de trabajos ajenos será calificado con 0 y notificado según el reglamento institucional.

📄 Declaración de autoría

Declaro que el presente trabajo es de mi autoría, que no se utilizó inteligencia artificial (ni otra herramienta generadora de contenido) ni copia de terceros para su elaboración, y que conozco y acepto las sanciones establecidas en el reglamento institucional en caso de incumplimiento.



Firma del estudiante

Docente